



Características arquitectónicas

IIC2173 - Arquitectura de sistemas de software

Departamento de Ciencia de la Computación

Escuela de Ingeniería – PUC

Nicolás Acosta – Gerardo Crot

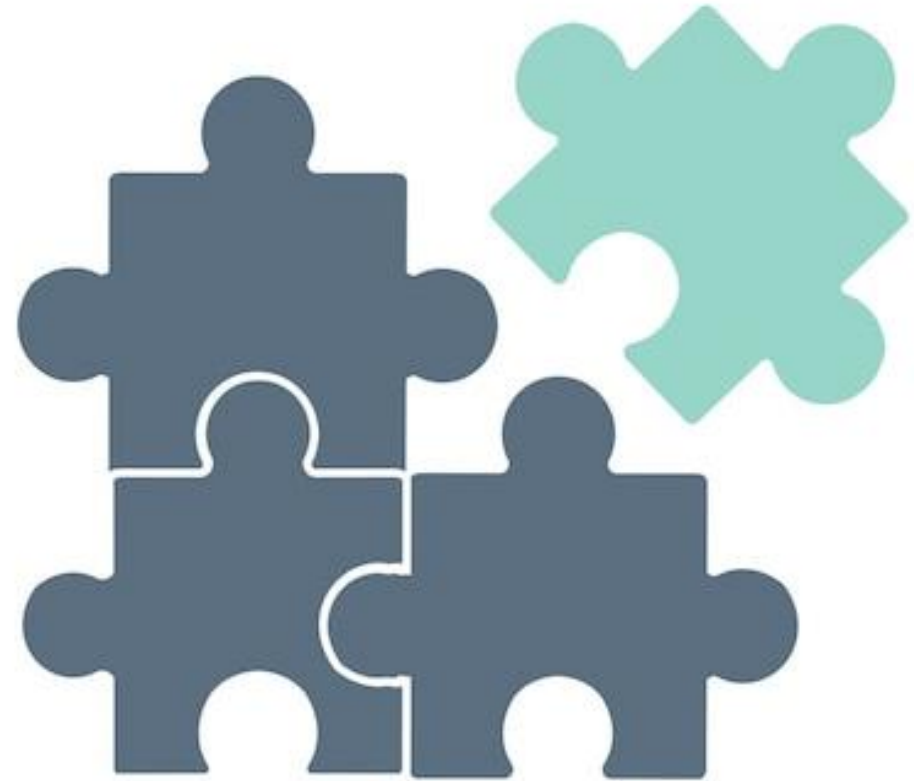
Repaso

- Decisiones de atributos
- Atributos de calidad estructurales
- Atributos de calidad operacionales



Requisitos transversales

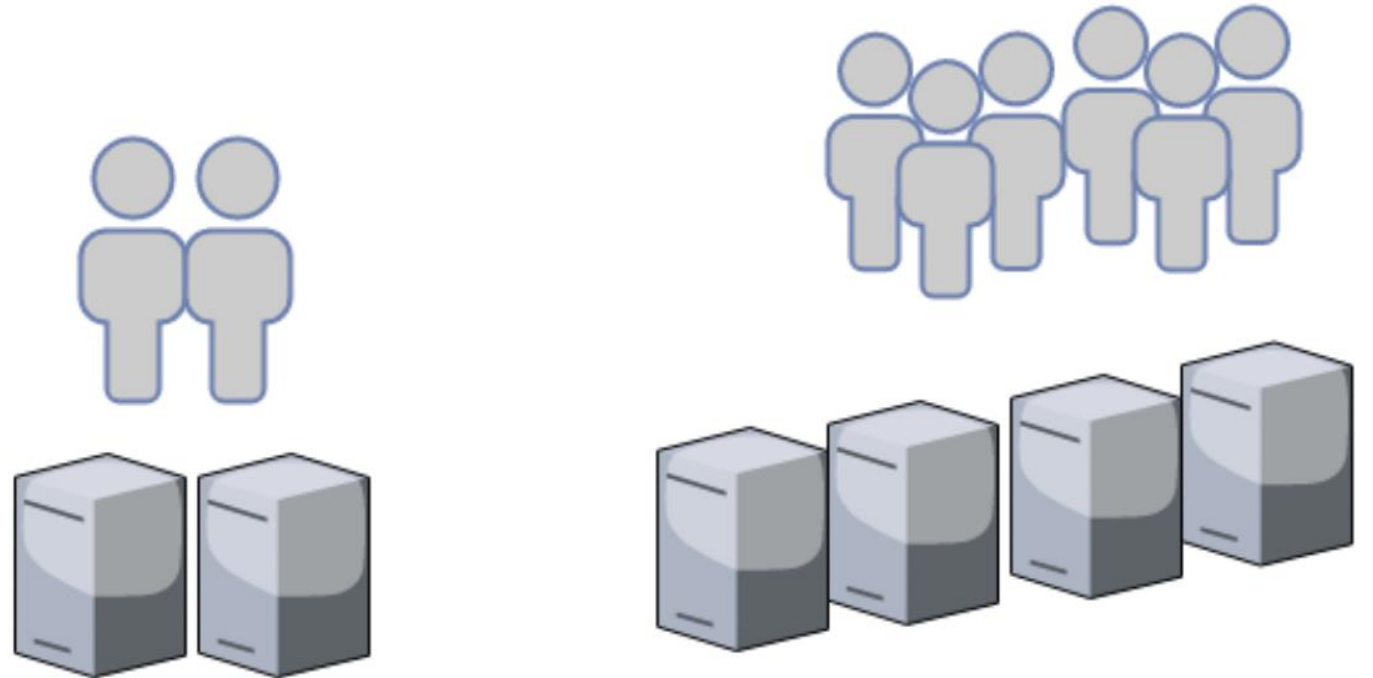
- Involucran varias partes de un sistema a varios niveles
- Son aquellos atributos que **afectan tanto la parte estructural como operacional**
- Más difíciles de categorizar
- Incorporan más elementos de dominio



Elasticidad

Capacidad de soportar aumentos “instantáneos” de demanda operacional, obteniendo recursos en función de las necesidades

- Requiere:
 - Estructura orientada
 - Capacidad operacional
 - Entorno compatible
- Usualmente disponible en proveedores *clouds*
- ¡No confundir con escalabilidad!



Seguridad

Grado en que el *software* protege la información y acceso de los datos

- Completamente transversal
- Involucra a diversos actores y sistemas
- Conceptos clave:
 - *Authorization/Authentication*
 - AC
 - IAM
- Control de acceso: RBAC/CBAC
- Guiado por principios



Seguridad: Algunos consejos

- Mínimo privilegio
- Defensa en profundidad
- *Fail-safe*
- Separación de responsabilidades
- Diseño abierto



Seguridad

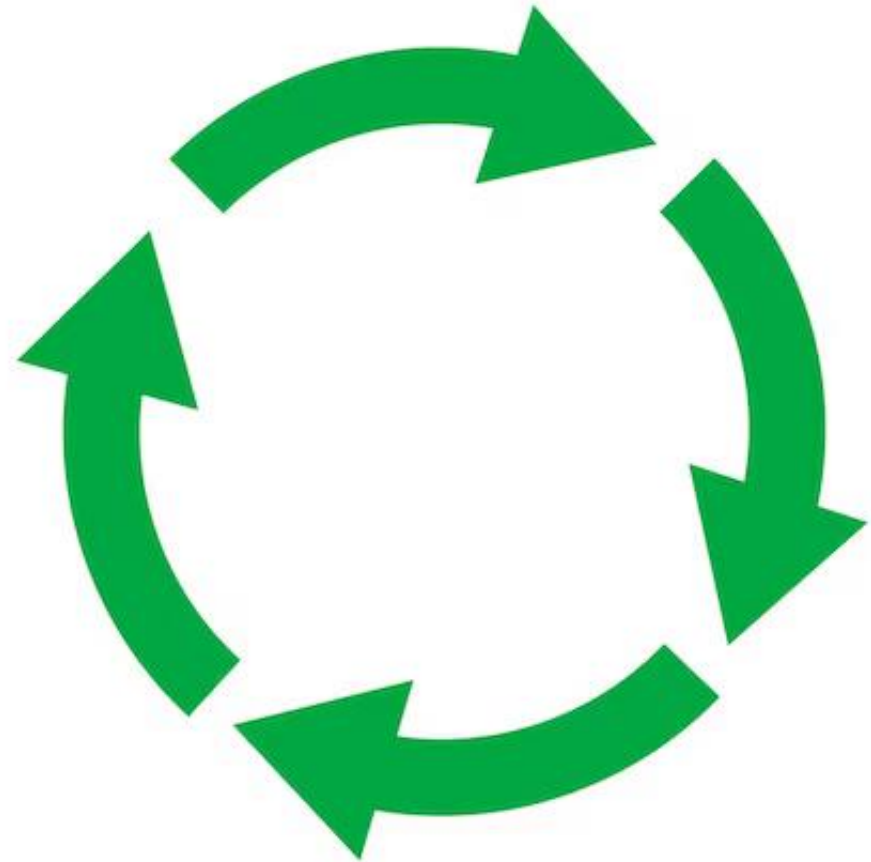
- Usualmente está presente pero no es máxima prioridad
- *Compliance*
- Data safety
- Debe ser manejada por expertos: En otro caso puede tener resultados catastrófico
- Requiere testeo constante
 - Pentest Xhat
 - Vulnerability scans
 - Protección/educación de usuarios



Reusabilidad

Capacidad de un sistema o componente de **volver a ocuparse en otros sistemas** más grandes

- Funcionalidad compartida
- ¡Precaución! Un exceso de reusabilidad puede provocar un fracaso arquitectónico



Soporte

Capacidad de **responder a requerimientos de modificación, mantención y extensión puntual del sistema**

- Relacionado a mantenibilidad
 - Operado por humanos
- Requiere una estructura sólida y capacidad operacional
- Asociado a métricas
 - Tickets/hora
 - SLAs



Eficiencia

Capacidad de cumplir las metas de negocio con el **menor uso de recursos posible**

- Altamente dependiente de otros NFR
- Muy medible, con métricas muy contextualizadas
 - ¿Es bueno que una app ocupe 1GB de RAM?
 - ¿Por qué mi aplicación aumenta tanto el uso de la CPU del servidor?



Usabilidad

Nivel de **entrenamiento** requerido por los usuarios para lograr sus metas con la aplicación/solución

Fundamentals of Software Architecture

- Bastante vago, pero debe considerarse: más difícil de medir
- Podemos requerirlo en prácticamente todas las apps con interacción, **hay que saber cuándo priorizarlo**
- Fuertemente influenciado por muchos otros atributos





¿Cómo aprovechamos los Atributos de Calidad?

- Recordar que ven las “funcionalidades globales” de la aplicación
- Cumplir todos los atributos sería lo ideal, pero no siempre es factible
- Un atributo sin métrica pierde sustentabilidad
- Demasiados atributos limitan la capacidad de acción, pocos limitan utilidad
 - Debemos saber cuándo elegirlos y cuáles priorizar



Introducción a diagramación

IIC2173 - Arquitectura de sistemas de software

Departamento de Ciencia de la Computación

Escuela de Ingeniería – PUC

Nicolás Acosta – Gerardo Crot

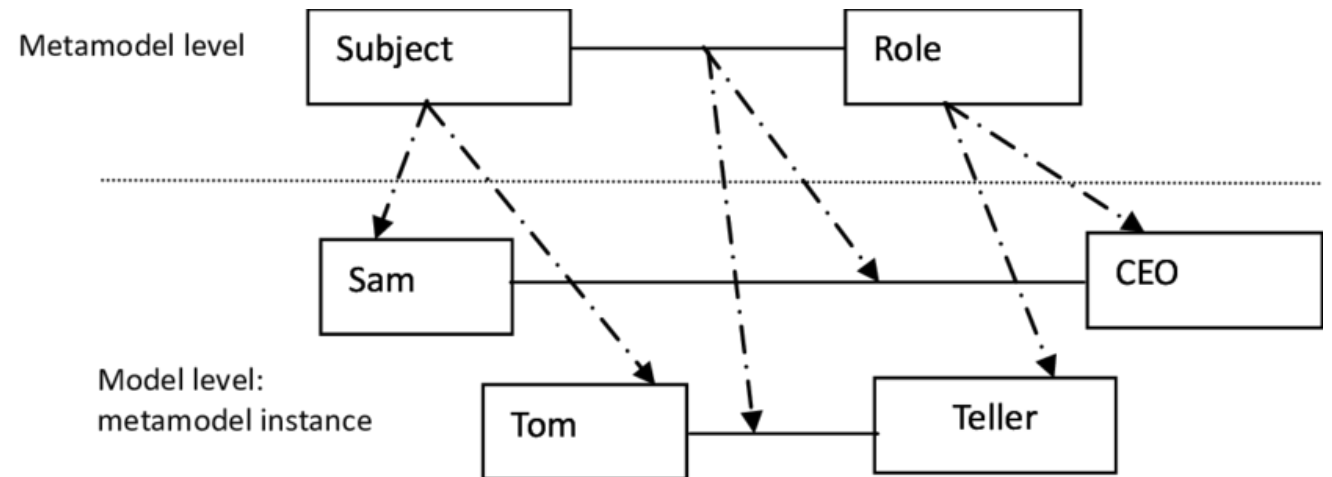
Diagramación

- Forma de **transmisión de conocimiento clave y eficiente**
- Debe ir acompañado de un relato de funcionamiento
- Debe estar enfocado en una audiencia específica: Ojo con developer experience!



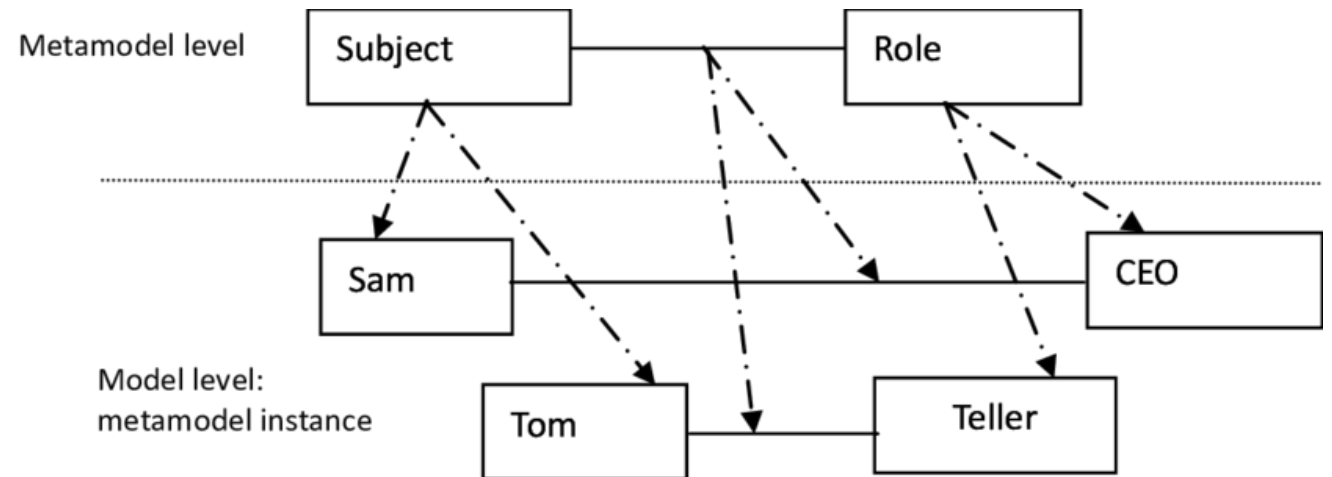
Diagramación

- Definimos dos niveles para el diseño:
 - Modelo
 - Representación abstracta de un sistema
 - Meta-modelo
 - Representación abstracta para entender el modelo
 - Modelo del modelo (varios niveles de abstracción)



Diagramación

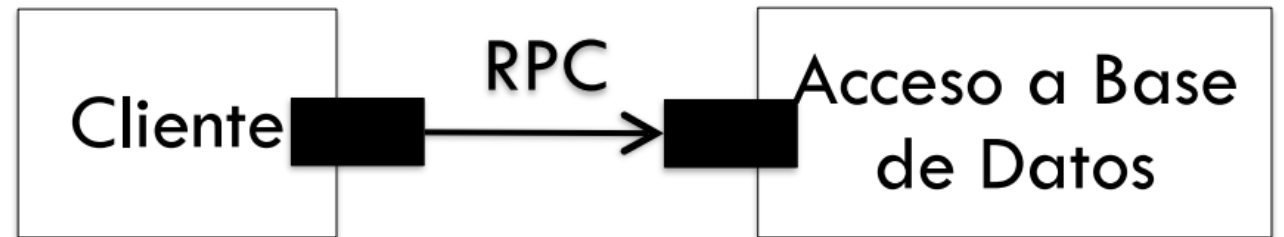
- ¿Por qué un meta-modelo?
 - Permiten definir lenguajes de modelación que a su vez describen sistemas de software
 - Estructura o sintaxis abstracta del lenguaje
 - Semántica formal o semi-formal
 - Requieren una sintaxis concreta (notaciones textuales, gráficas, etc.)
- Necesitamos saber cómo entender los modelos



Architecture Description Language (ADL)

Lenguaje de modelación específicamente diseñado para describir software o arquitecturas de software

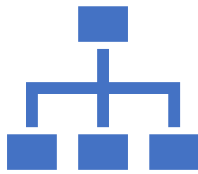
- Representación formal (textual, gráfica)
- Legible por humanos y máquinas (ACME, UML, etc), incluidos stakeholders y sistemas de comprobación
- Analizable sobre sus propiedades arquitectónicas (consistencia, completitud, desempeño, etc)
- Permiten generación de código



Especificación escrita en ACME:

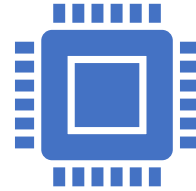
```
System CS_System = {  
  Component customer = {Port send-request}  
  Component databaseAccess = {Port receive-request}  
  Connector rpc = {Roles {caller, callee}}  
  Attachments : {  
    customer.send-request to rpc.caller;  
    databaseAccess.receive-request to rpc.callee  
  }  
}
```

Tipos de diagramación



Vista Organizacional

Estructura de la organización



Vista de sistemas

Interconexiones de sistemas y subsistemas entre las organizaciones involucradas y los sistemas externos

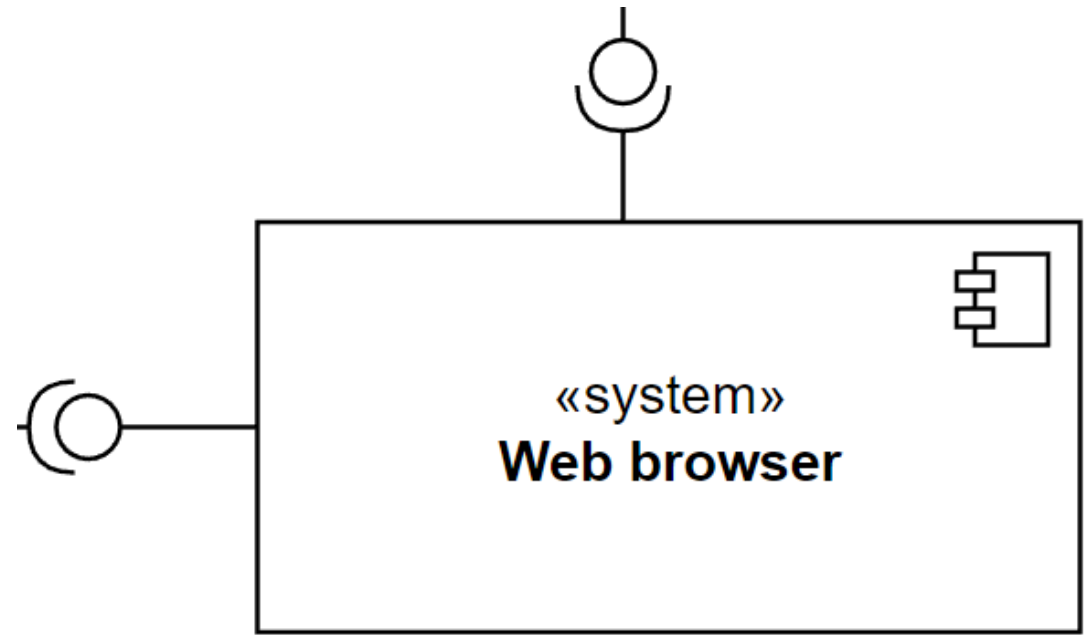


Vista de componentes

Aspectos funcionales y no funcionales de los componentes

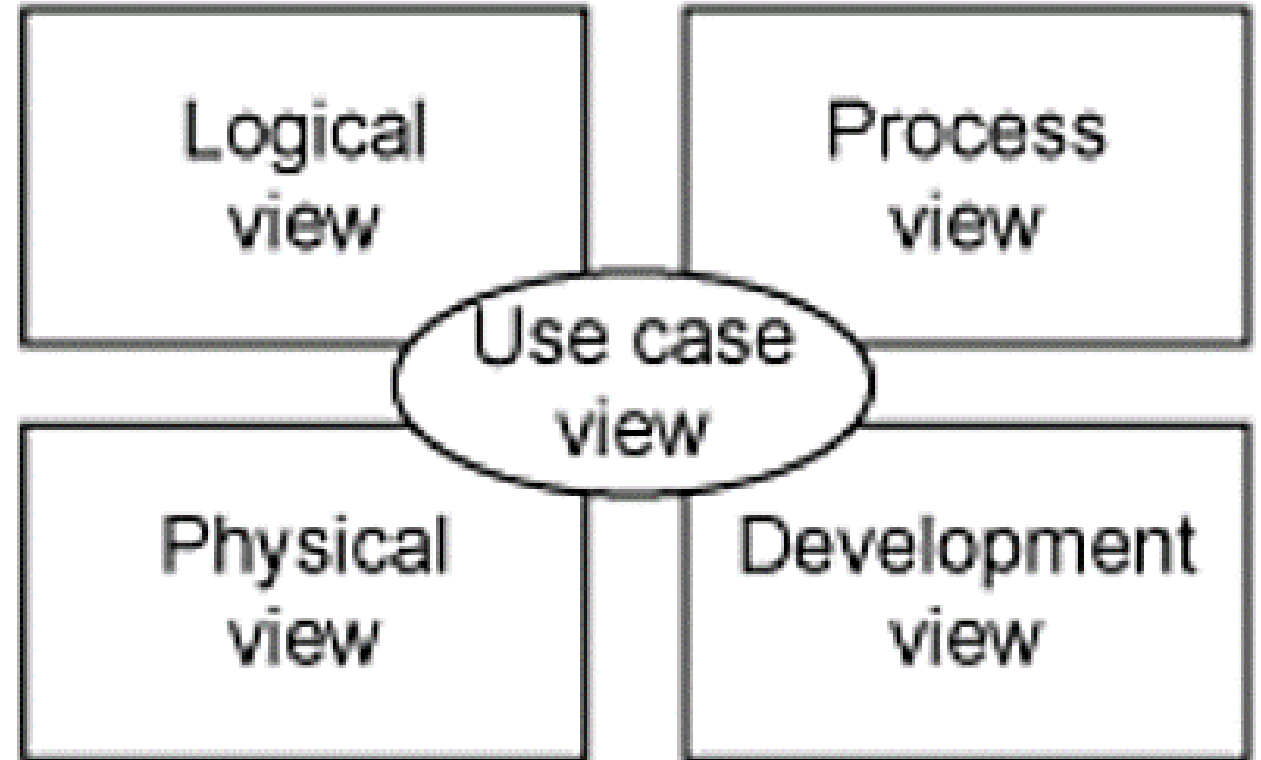
Conceptos principales

- Componentes
 - Especificación funcional o no funcional de los elementos y sus interfaces
- Conectores
 - Indicaciones de cómo interactúan los componentes
- Configuraciones arquitectónicas
 - Describe la estructura arquitectónica indicando como se conectan componentes y conectores
- Existen **distintos estándares**



Vista de modelo 4+1

- Fundamental
- Las decisiones arquitectónicas se basan en casos de uso y escenarios del sistema
- Pueden usarse diversos tipos de diagrama para cumplir con cada una de las partes

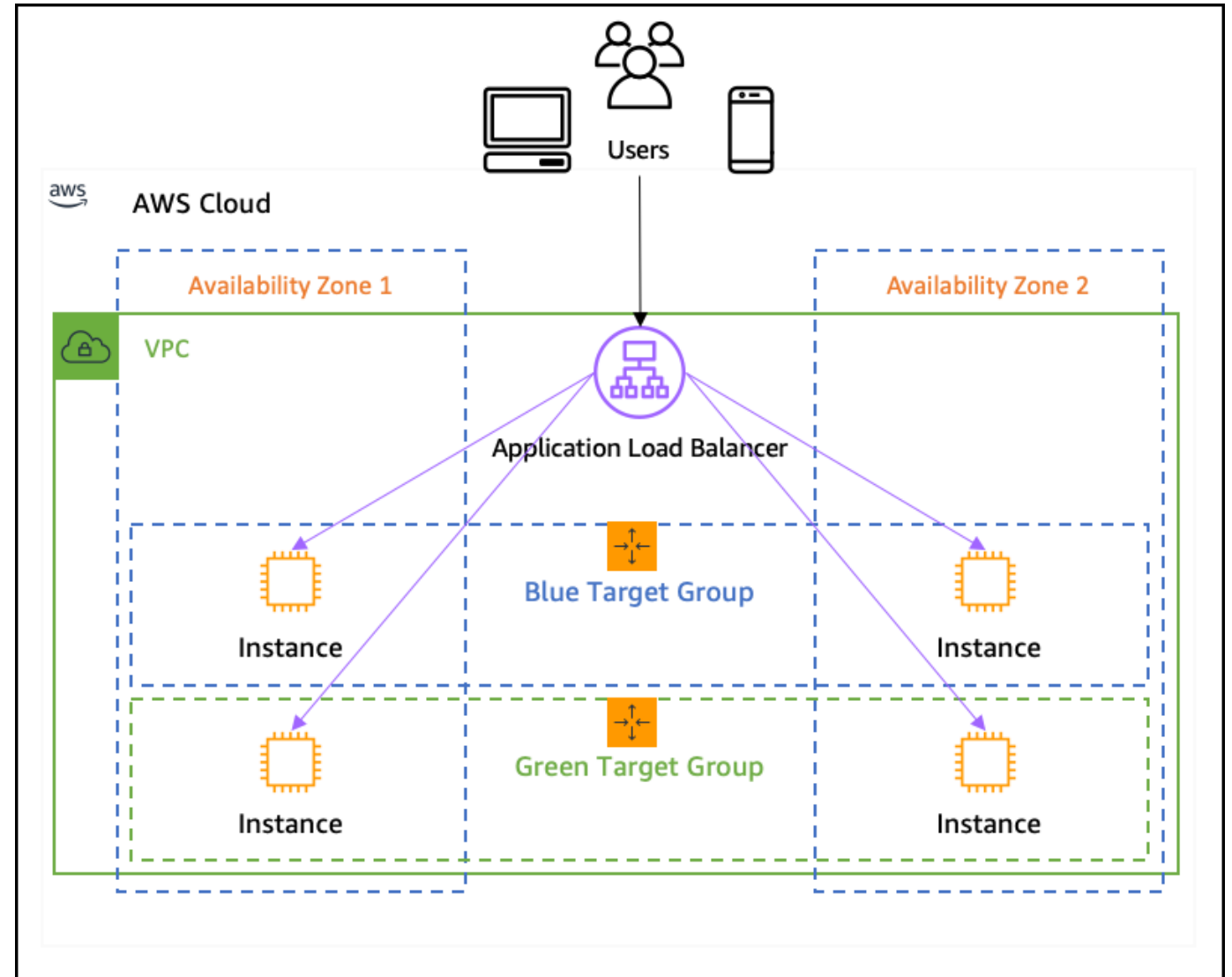


Vista de modelo 4+1

- Vista lógica
 - Abstracción del sistema: Sistema de clases y objetos
 - Diagramas de clase, secuencia y colaboración
- Vista de desarrollo
 - Estructura del software en términos de módulos, archivos y paquetes del sistema
 - Diagrama de paquetes (componentes)
- Vista de proceso
 - Procesos del sistema y su interacción
 - Diagramas de actividad

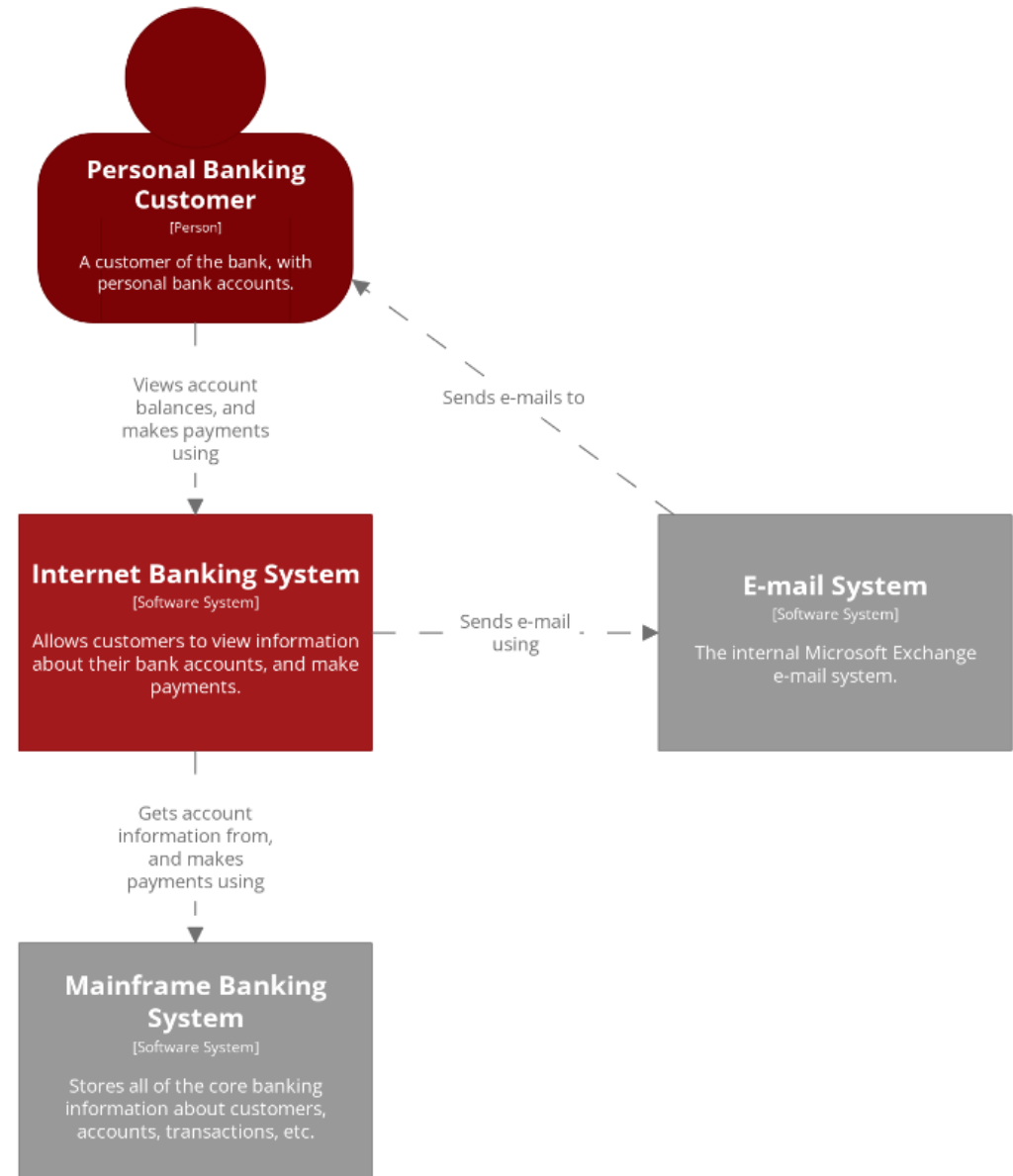
Vista de modelo 4+1

- Vista física
 - Como se instala el sistema y como se ejecuta (en una red de computadores)
 - Diagramas de despliegue (*deployment*)
- (+1) Vista de Escenario
 - Presenta como los actores usan la aplicación
 - Diagramas de casos de uso



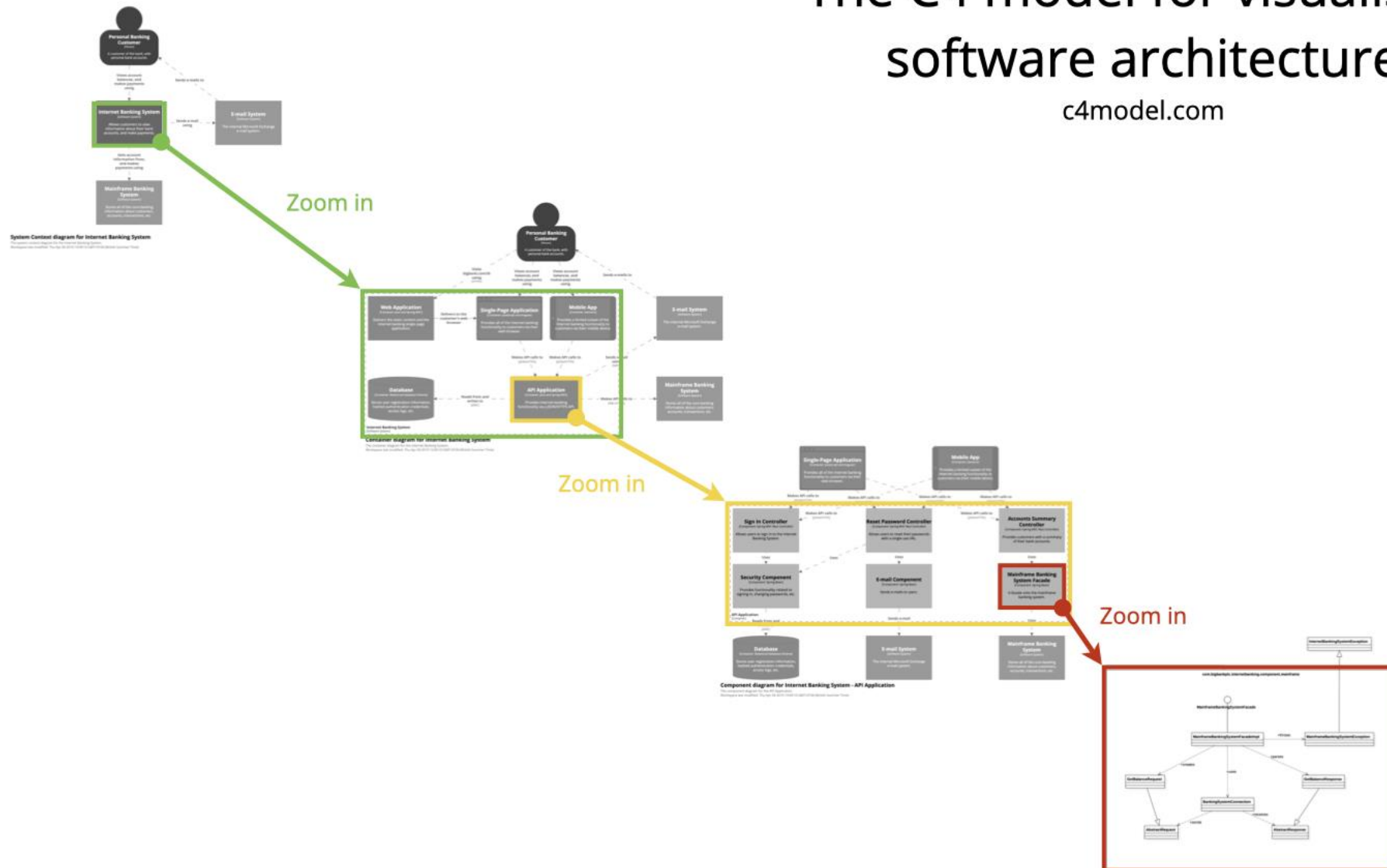
C4

- Sencillo de aprender, fácil de usar
- Menos formal y estructurado que UML pero muy útil
- *Trade-off*: más fácil de iterar vs menos rigurosidad



The C4 model for visualising software architecture

c4model.com



Level 1
Context

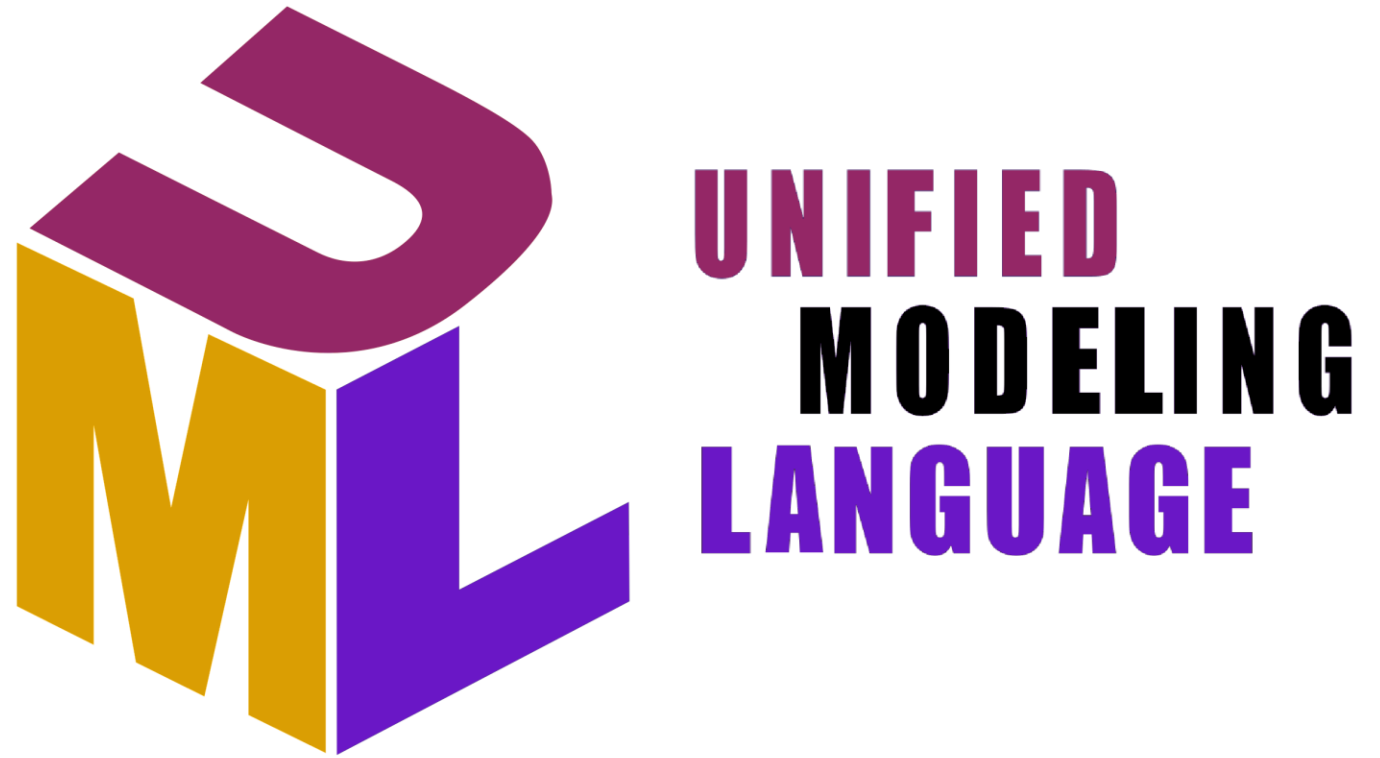
Level 2
Containers

Level 3
Components

Level 4
Code

UML

- Creado por el grupo OMG (*Object Management Group*) circa. 2000
- Enorme modelo, pero usable en varios contextos
- Comprensible por varias audiencias
- Bastante complejo en sus fundaciones



UML

- Infraestructura UML
 - Define elementos usados en el meta-modelo de UML y en la superestructura
- Superestructura UML
 - Define el meta-modelo de UML (define el lenguaje)
 - <https://www.omg.org/spec/UML/2.2/Superstructure/PDF> (740p)



OMG Unified Modeling Language™ (OMG UML), Superstructure

Version 2.2

OMG Document Number: formal/2009-02-02

Standard document URL: <http://www.omg.org/spec/UML/2.2/Superstructure>

Associated Files:

<http://www.omg.org/spec/UML/20080501/Superstructure.cmo>

<http://www.omg.org/spec/UML/20080501/uml-L0-model.xmi>

<http://www.omg.org/spec/UML/20080501/uml-LM-model.xmi>

<http://www.omg.org/spec/UML/20080501/uml-L1-model.xmi>

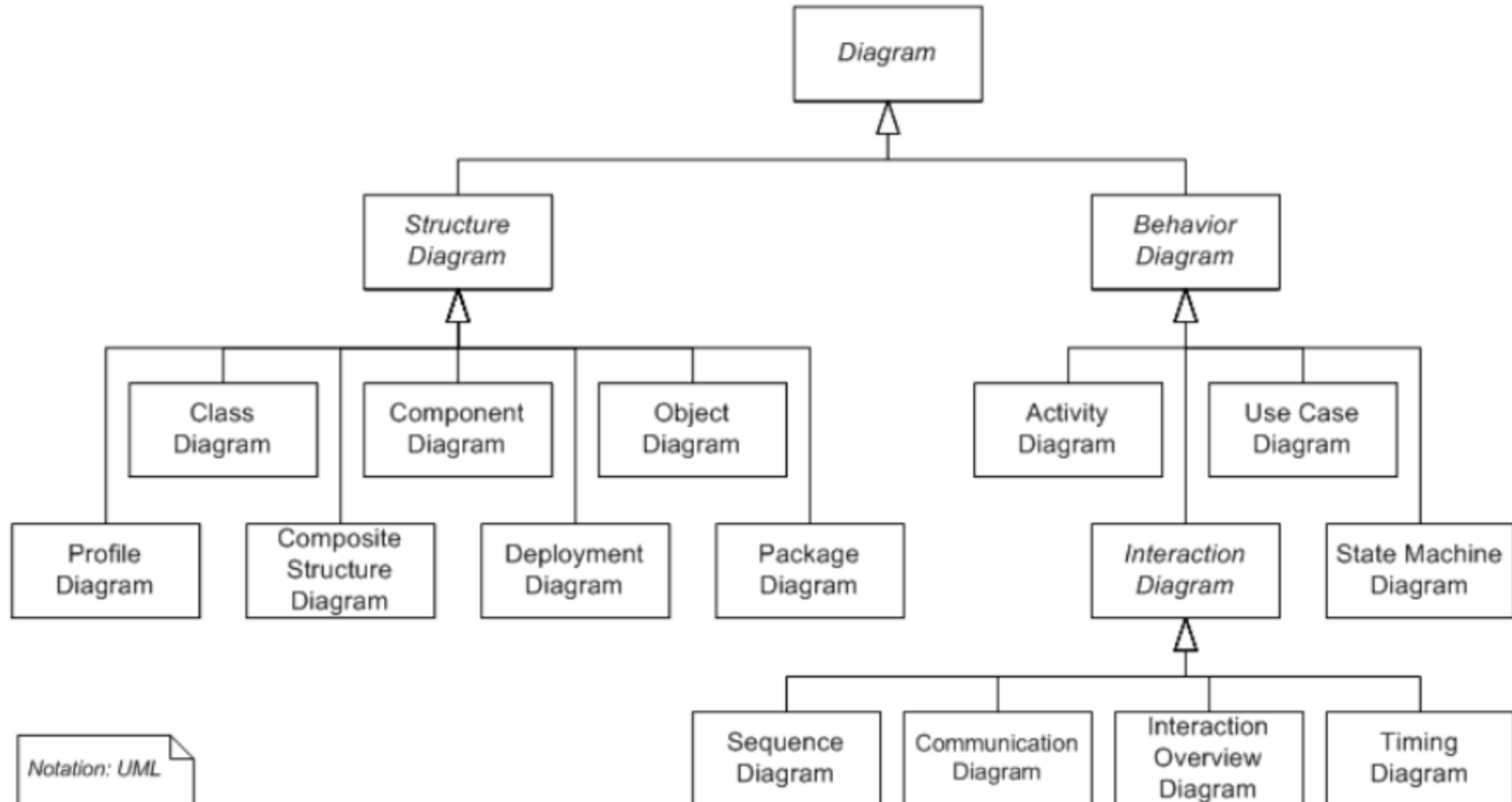
<http://www.omg.org/spec/UML/20080501/uml-L2-model.xmi>

<http://www.omg.org/spec/UML/20080501/uml-L3-model.xmi>

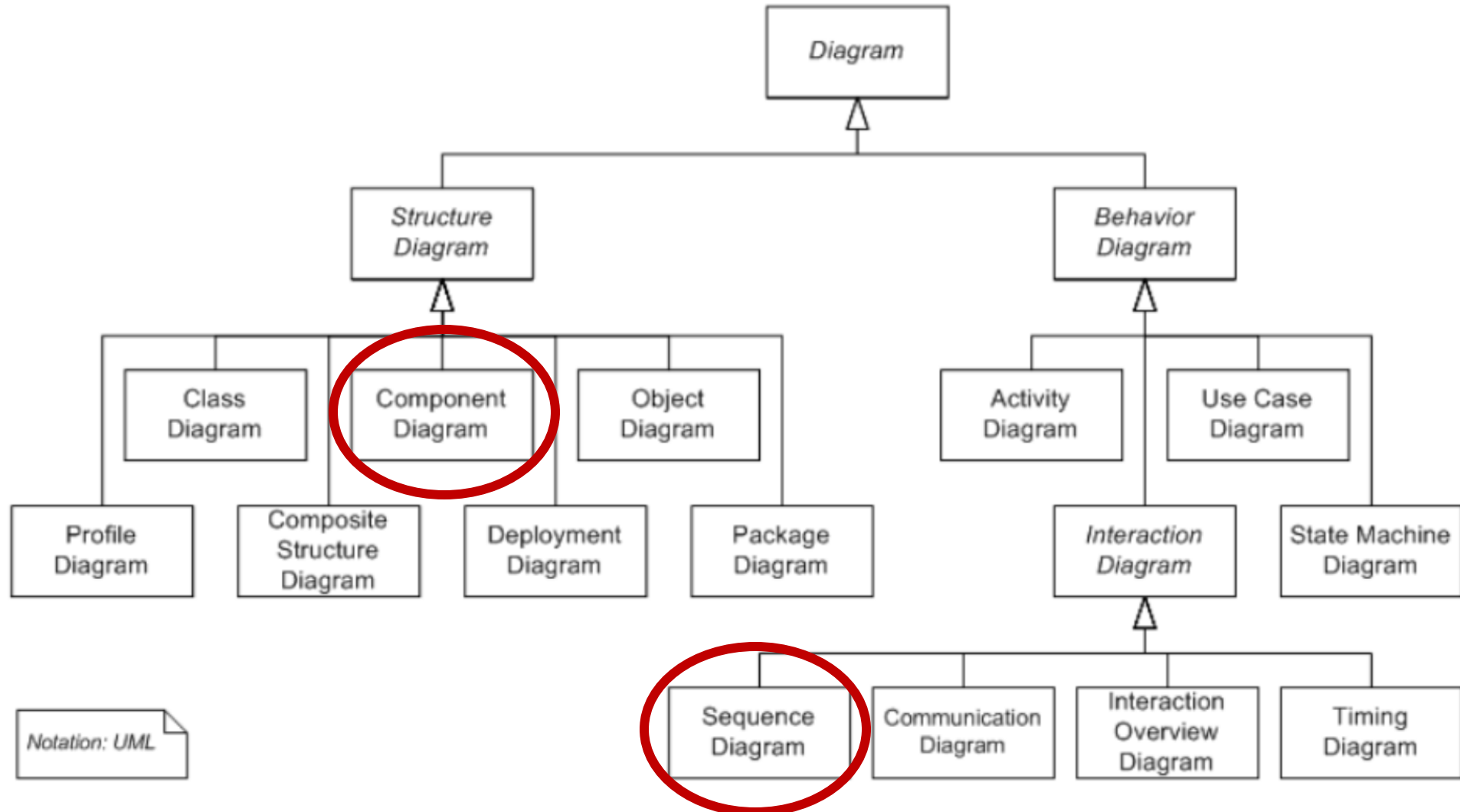
Normative machine-readable files: ptc/08-05-07, ptc/08-05-08, ptc/08-05-09, ptc/08-05-10, ptc/08-05-11, ptc/08-05-12.

Version 2.2 is a minor revision to the UML 2.1.2 specification. It supersedes formal/2007-11-02.

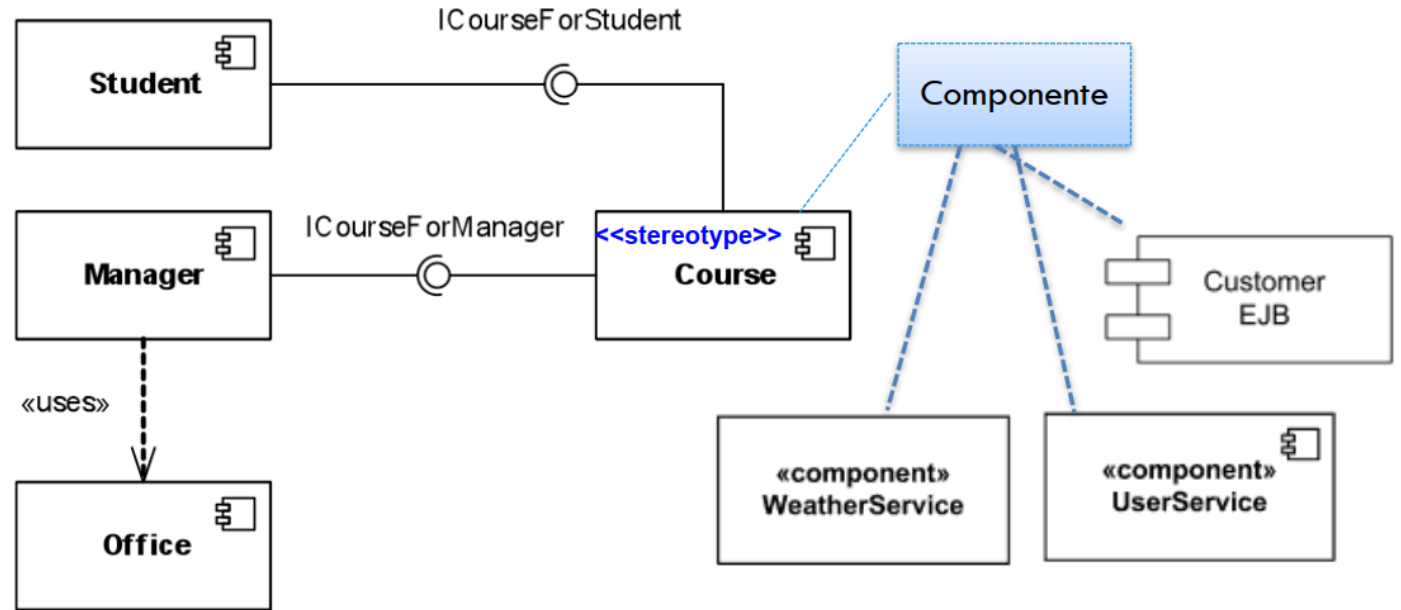
UML: Variantes



UML: Variantes

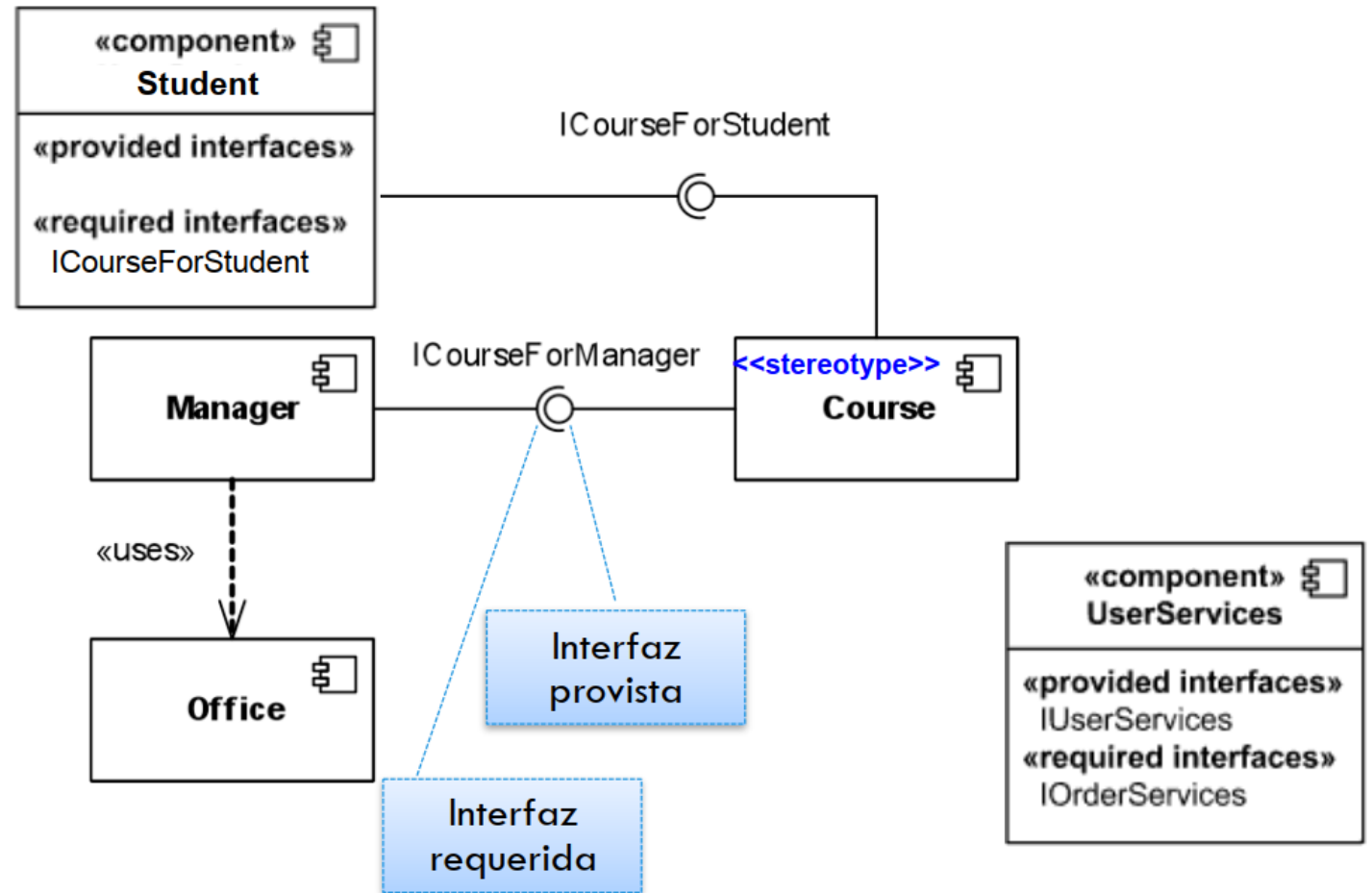


Diagramas UML clave: Componentes



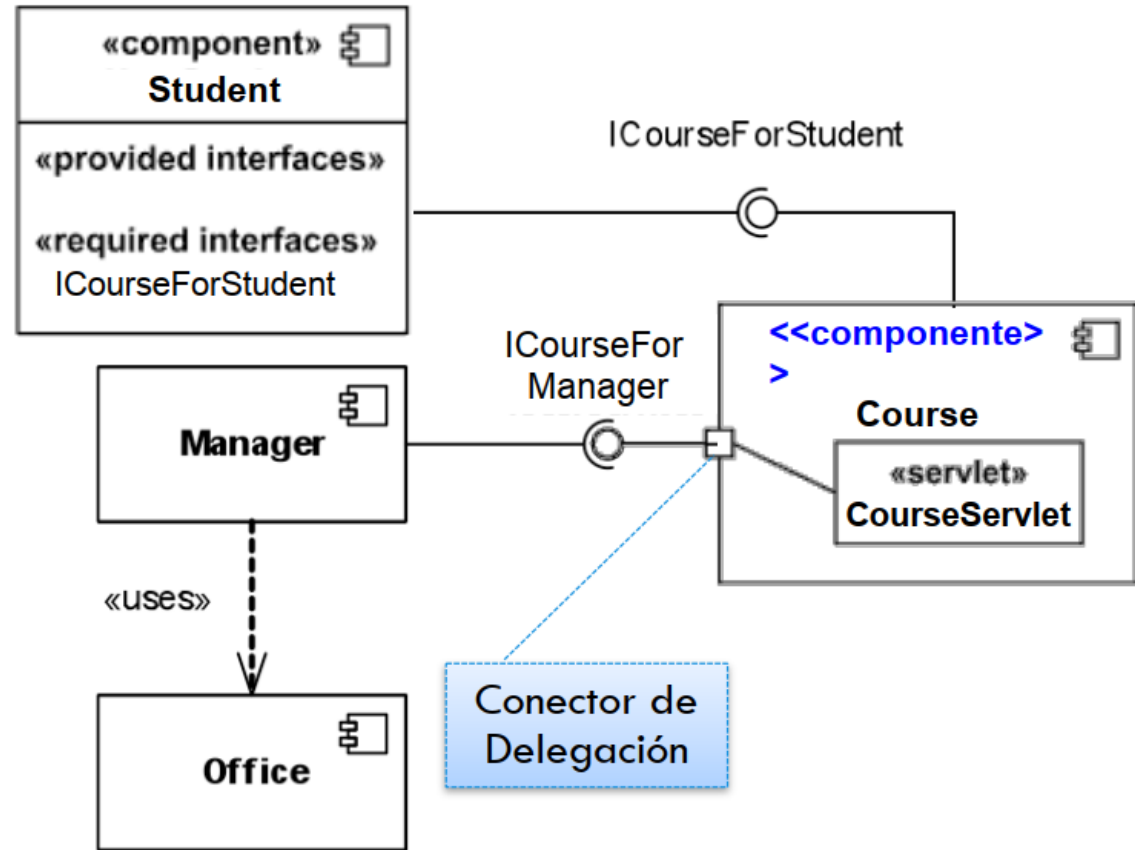
Estereotipos estándar para componentes: **«subsystem»**, **«process»**, **«service»**, **«specification»**, **«realization»**, **«implement»**

Diagramas UML clave: Componentes



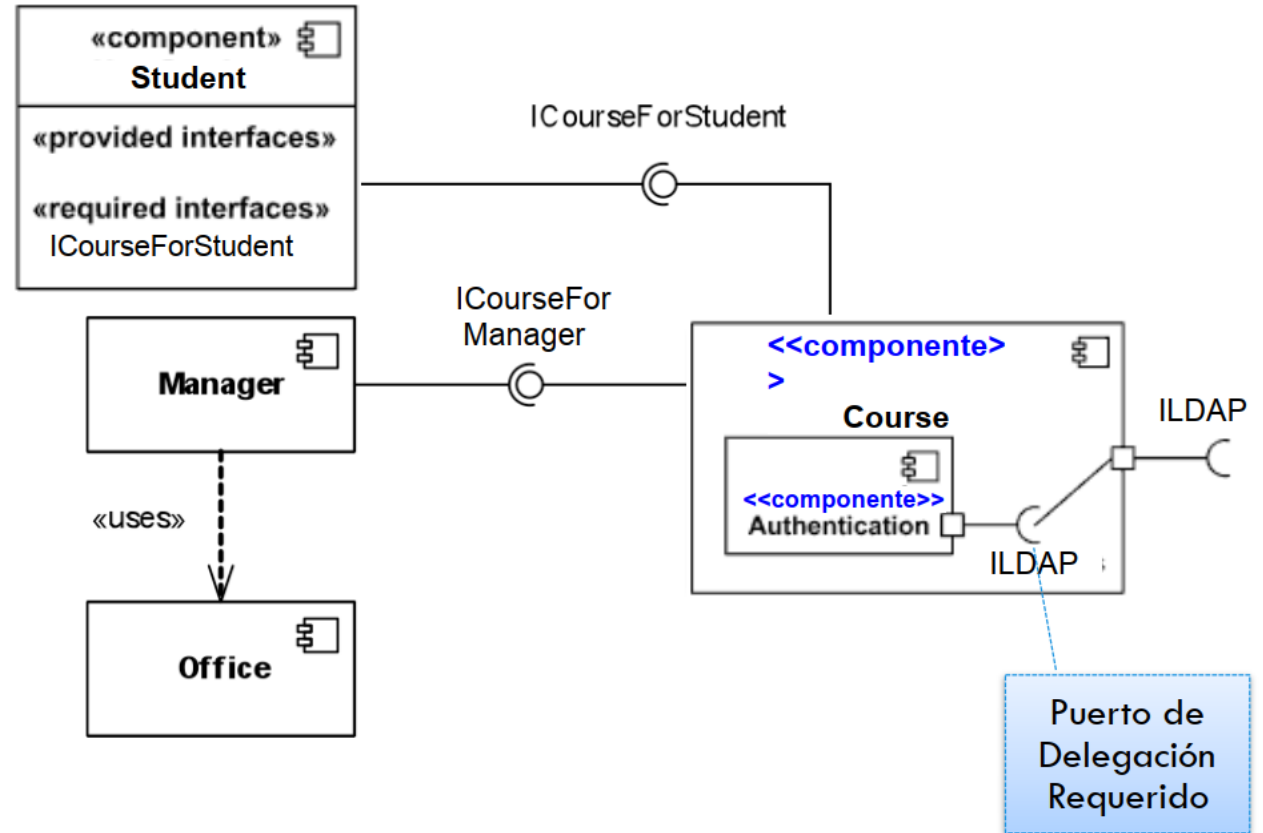
Interfaces provistas e interfaces requeridas gráficamente y detalle textual

Diagramas UML clave: Componentes



Conectores de delegación: Descomposición jerárquica de funcionalidad

Diagramas UML clave: Componentes



Conectores de delegación de salida: Delegación de un puerto interno a un puerto de delegación de salida

¿Por qué usar UML?



Estándar en la industria



Bueno para ver y mostrar soluciones rápidamente



Refleja las ideas más fielmente



Se logra un mayor detalle de una implementación

Premisas de diagramación

- No hay un solo tipo de diagrama
- No hay un sistema para resolver todo
- Recordar analogías apropiadas:
 - Imágenes
 - Vistas construcción

TYPE	PURPOSE
Wireframes / UX / UI design	Shows exactly what the user interface looks like, typically as part of a transaction or interactions that a user will have with the system
Process map / decision tree / flow chart	Visual representation of a workflow or process. Often shows the various choices along with effects of those choices
Data flow	Shows how data flows from one system/service to the next
Data model	Shows tables, fields, keys, indexes, and data types in a data store
Deployment / physical diagram	Shows things like HA, DR, scaling, account / data center, networking, etc types of components
System landscape diagram	Bigger, higher level of a system diagram, that shows how many systems interact and integrate with each other
Conceptual diagram	Highest level of a functional diagram. Doesn't show technologies but more from a concept or generalized view
Logical diagram	Shows functional components and how they interact with each other

Premisas de diagramación

- Buscar un relato:
Se puede describir cada sistema o funcionalidad como una historia de investigación
- Sin asumir antes de tiempo
- Sin saltarse información
- Partir de alto nivel enfocando a piezas específicas

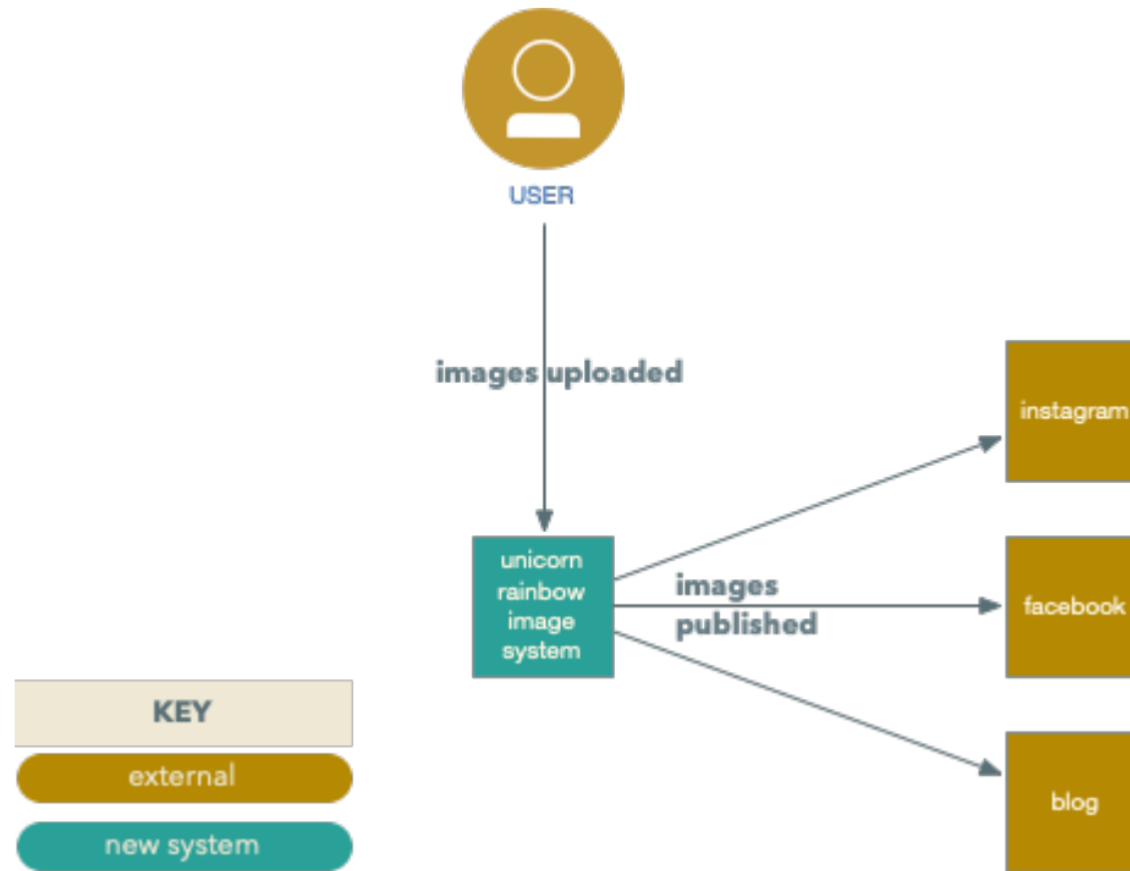
TYPE	PURPOSE
Wireframes / UX / UI design	Shows exactly what the user interface looks like, typically as part of a transaction or interactions that a user will have with the system
Process map / decision tree / flow chart	Visual representation of a workflow or process. Often shows the various choices along with effects of those choices
Data flow	Shows how data flows from one system/service to the next
Data model	Shows tables, fields, keys, indexes, and data types in a data store
Deployment / physical diagram	Shows things like HA, DR, scaling, account / data center, networking, etc types of components
System landscape diagram	Bigger, higher level of a system diagram, that shows how many systems interact and integrate with each other
Conceptual diagram	Highest level of a functional diagram. Doesn't show technologies but more from a concept or generalized view
Logical diagram	Shows functional components and how they interact with each other

Métodos clave de diagramación

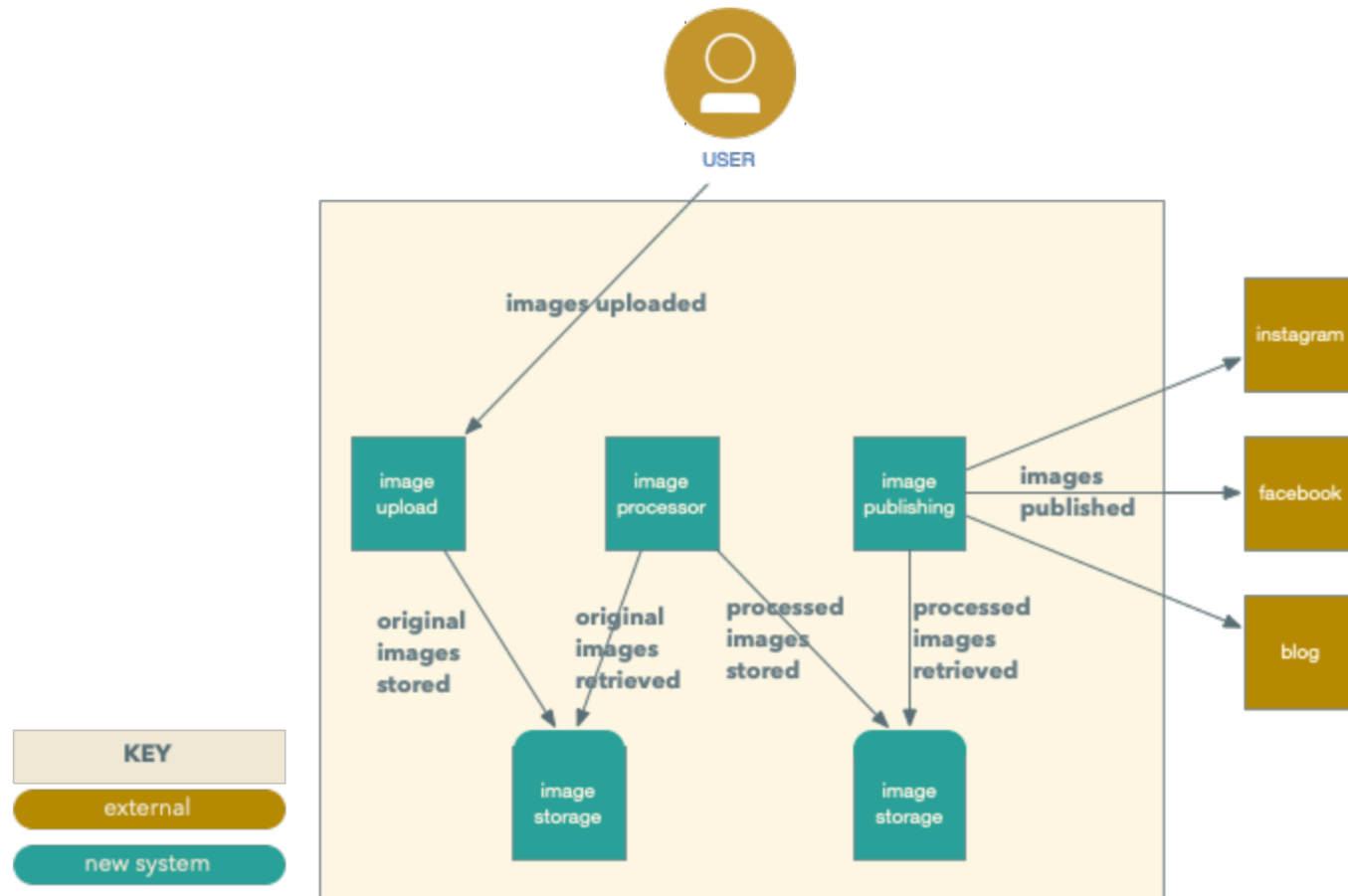


Inductivo: Detalle a lo general	Deductivo: General al detalle	Referencia
“Tengo que armar un chat en tiempo real”: Debe aplicar invocación implícita --- “Tengo que almacenar fotos”: hay que introducir un tipo de <i>storage</i> de objetos	“Debo cubrir 1000 usuarios por hora”: Debe usar un balanceador de carga --- “Mi sistema se puede dividir según lo estructuré”: Debo tener una Arquitectura orientada a servicios (SoA)	“He visto esto en otro lado...” Hay componentes reusables que se pueden usar en nuestros sistemas

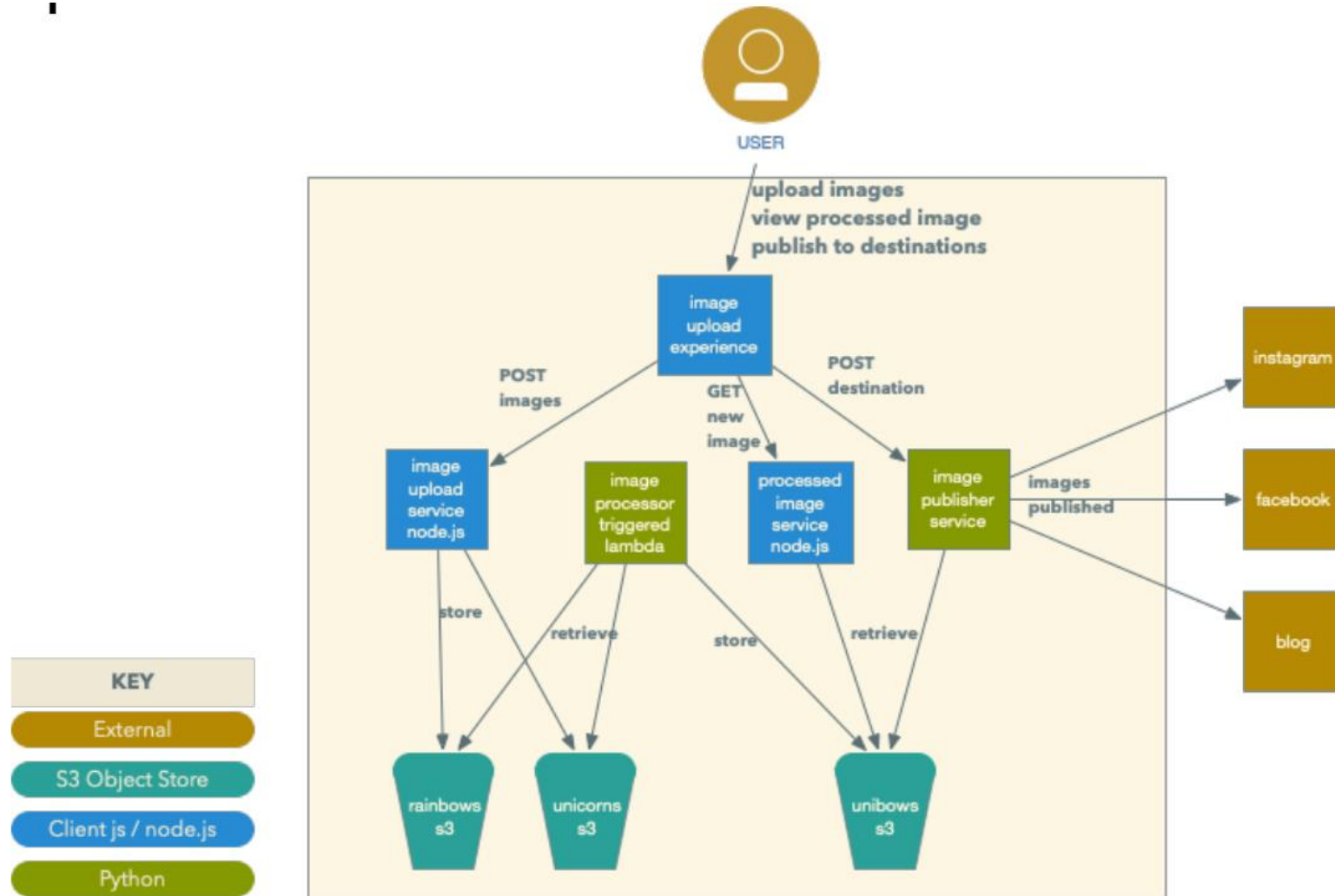
Ejemplo: Subir imágenes a redes multiples fuentes



Ejemplo: Container o componentes



Ejemplo: Componentes



Objetos que me permiten lograr la comprensión del problema

- Contextualización
- Core
- Especificaciones
- Detalles
- Supuestos
- Limitaciones





Introducción a Estilos y Patrones

IIC2173 - Arquitectura de sistemas de software

Departamento de Ciencia de la Computación

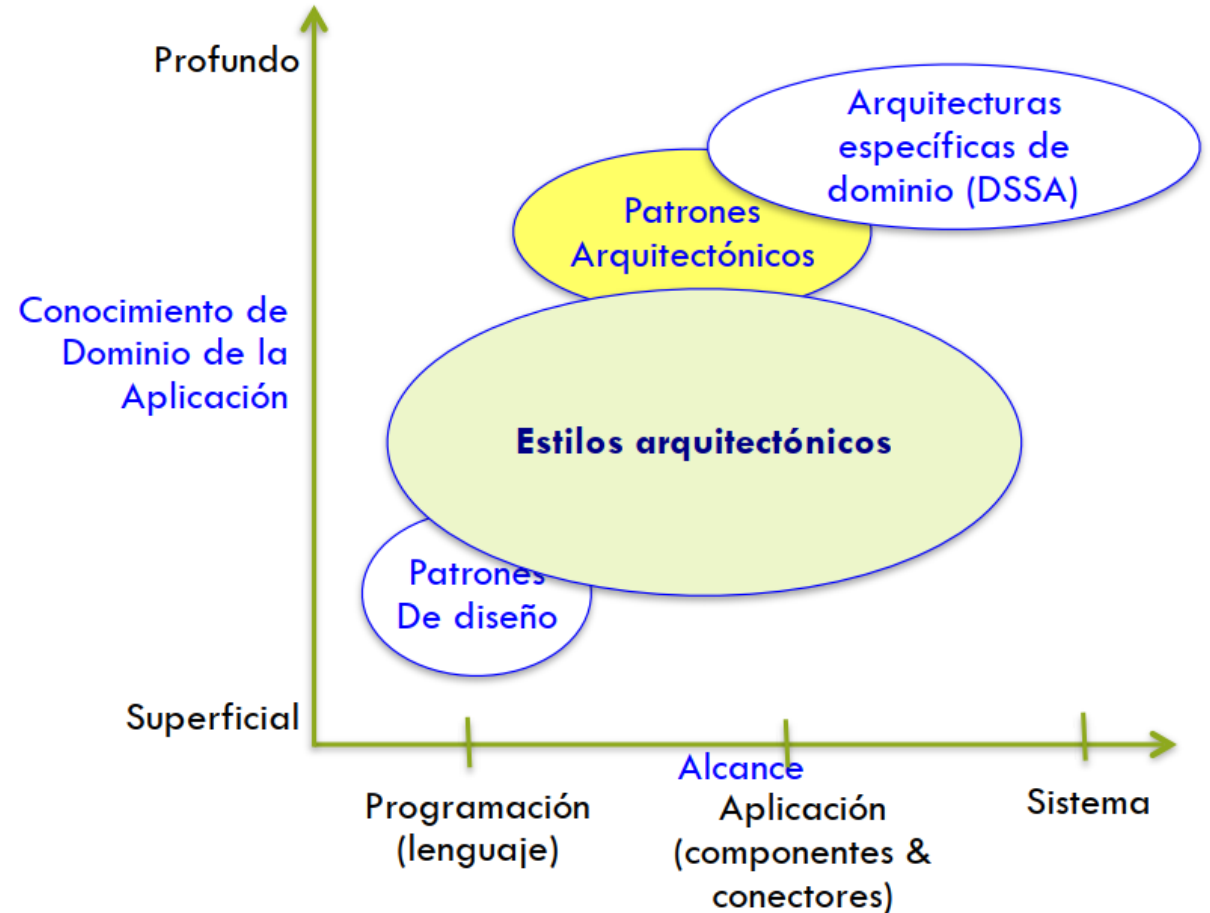
Escuela de Ingeniería – PUC

Nicolás Acosta – Gerardo Crot

Estilos arquitectónicos

Colección nombrada (con nombre) de decisiones de diseño arquitectónico que:

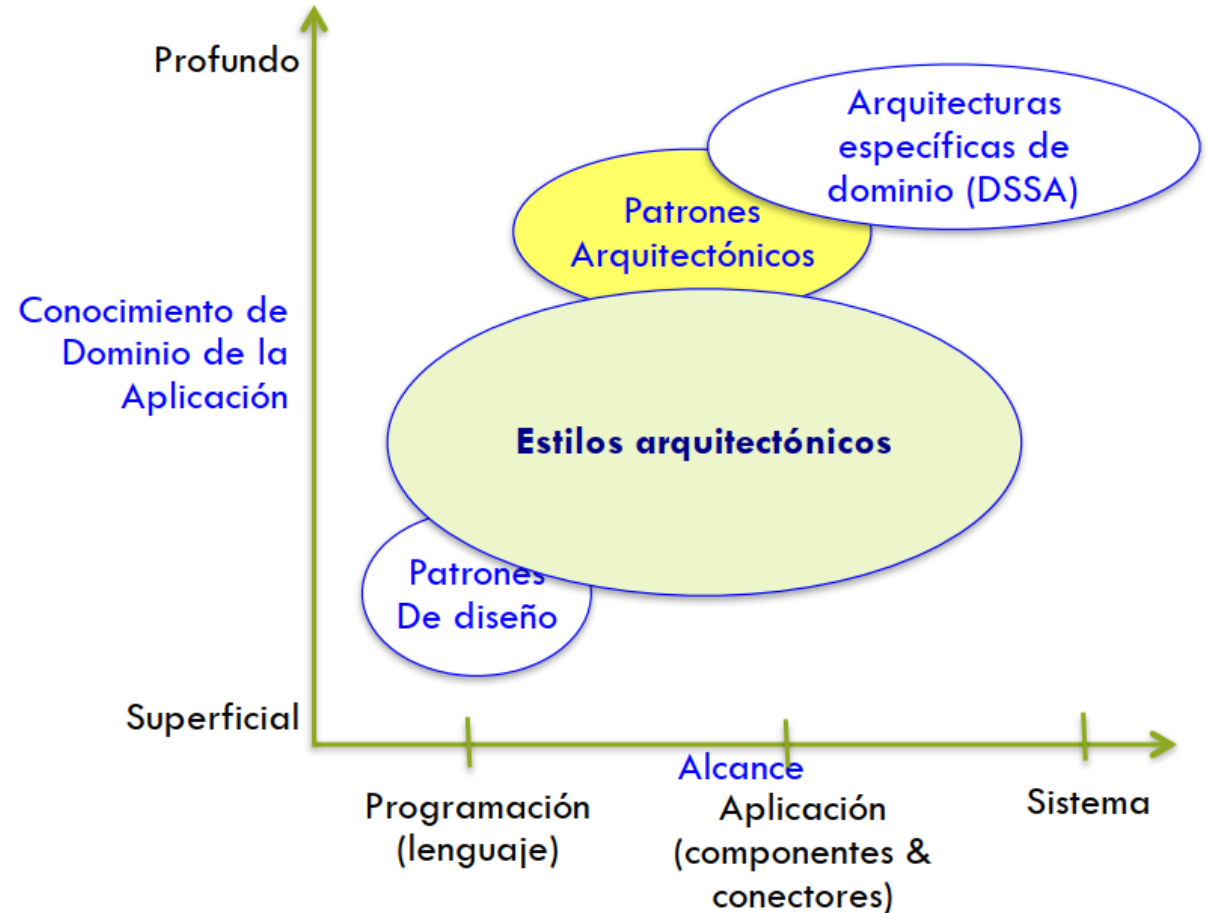
- Aplican a un **contexto de desarrollo dado**
- **Restringen las decisiones de diseño que son específicas a un sistema particular** dentro de ese contexto
- Promueven ciertos atributos de calidad
- Gobiernan
 - Código
 - Componentes
 - Requisitos
 - Decisiones



Patrones de Arquitectura

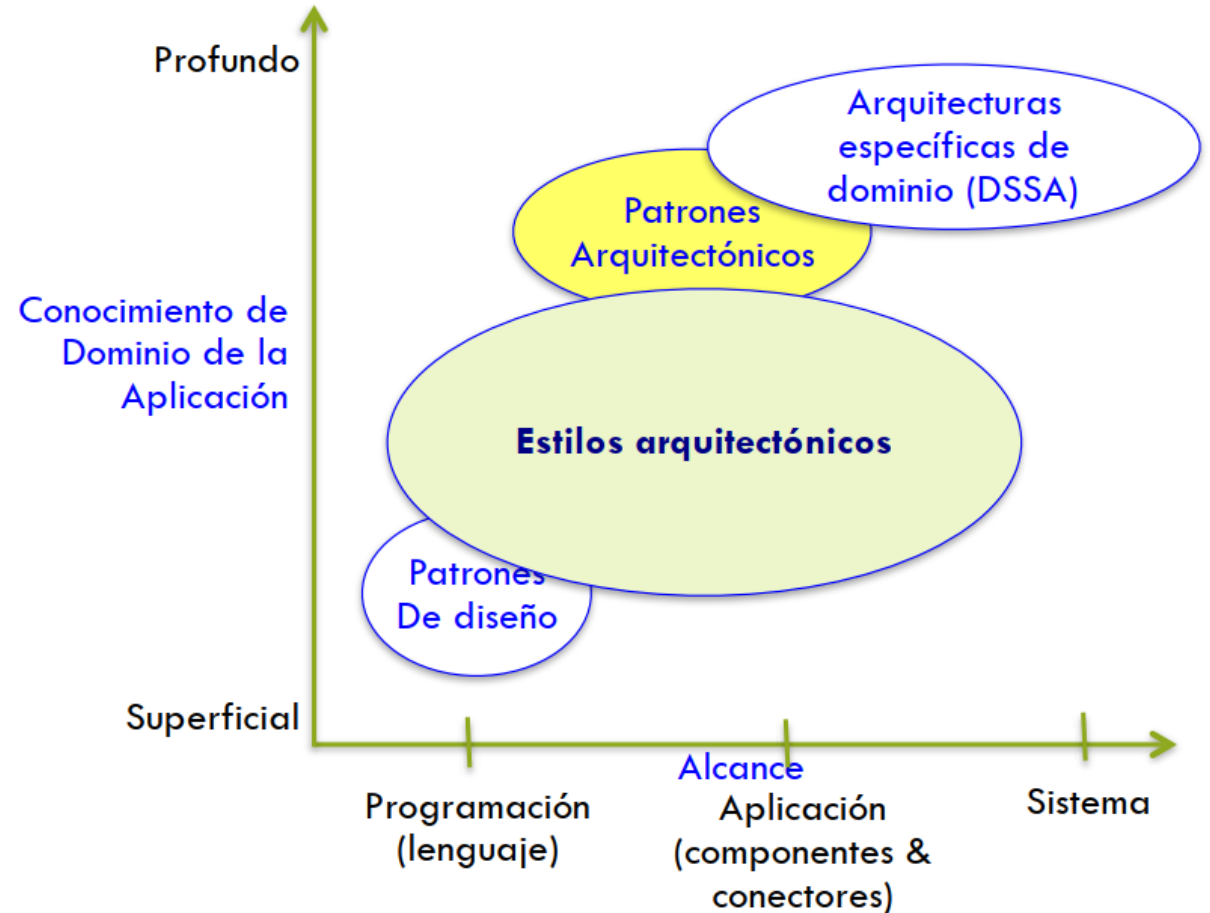
Colección nombrada de decisiones de diseño arquitectónico que:

- Aplican **a un problema de diseño recurrente**
- Se **parametriza según los contextos de desarrollo** en que aparece el problema



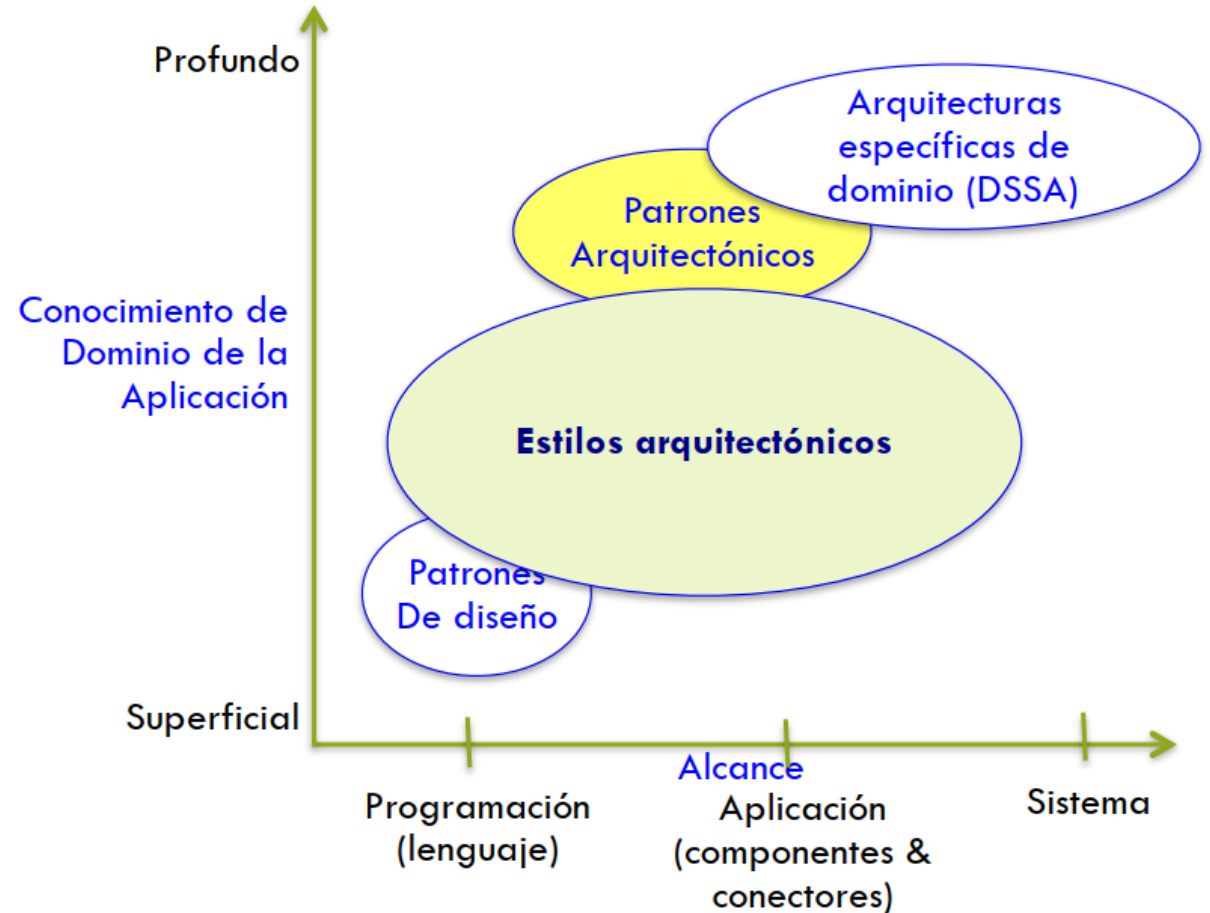
Diferencias entre estilo y patrón

- **Alcance:** Problema de diseño vs Contexto de desarrollo
- **Abstracción:** Fragmentos arquitectónicos parametrizados vs Requiere interpretación humana
- **Relación:** Un patrón puede aplicarse a sistemas que siguen varios estilos, un sistema que sigue un estilo puede usar varios patrones



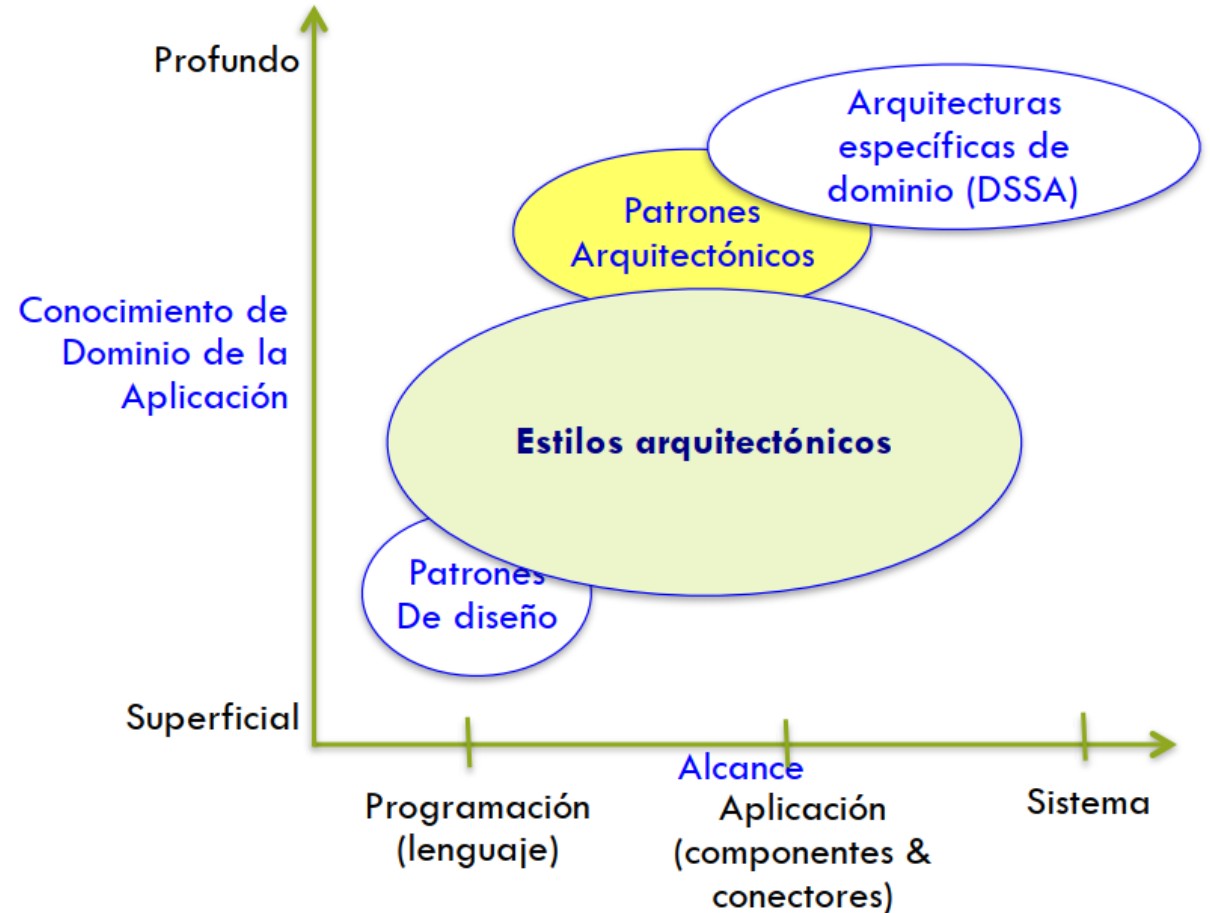
Ventajas de estilos y patrones

- Reuso de diseños probados
 - Soluciones bien entendidas aplicadas a nuevos problemas
 - Facilidad de composición
- Genera soluciones más maduras, establecidas
- Lenguaje compartido mejora comunicación
 - Ej: "Cliente-Servidor" ya indica cómo es el sistema
- DRY, se apoya en convenciones



Ventajas de estilos y patrones

- Promueven interoperabilidad
 - Debido a la estandarización del estilo y su mayor comprensión
- Análisis específico al estilo
 - Permite restringir el espacio de diseño y justificar decisiones más fácilmente
- Visualizaciones
 - Figuras específicas al estilo calzan con el modelo mental "ingenieril"





Qué queremos evitar:
Big ball of mud

Qué
queremos
evitar:
Vibes on
prod



Vibe
coding



Vibe
debugging

Monolito: una primera arquitectura

- Sistema contenido en una sola quanta
- Un bloque de software, una base de datos
- Alta cohesión, acoplamiento estático, aunque aprecia modularidad
- Implicancias dependientes del estilo, no es inherentemente malo (debatible)



Estilos comunes

1. Influenciados por el lenguaje

- 1.1 Main y sub-rutinas

- 1.2 Orientado al objeto

2. En capas

- 2.1 Máquinas virtuales

- 2.2 Cliente - Servidor

3. Microkernel

4. Flujo de datos

- 4.1 Por Lotes, secuencial

- 4.2 Tubería y Filtro (Pipe & Filter)

5. Memoria compartida

- 5.1 Pizarra

- 5.2 Basado en reglas

6. Intérprete

- 6.1 Intérprete

- 6.2 Código móvil

7. Invocación implícita

- 7.1 Basado en eventos

- 7.2 Publisher – Subscriber

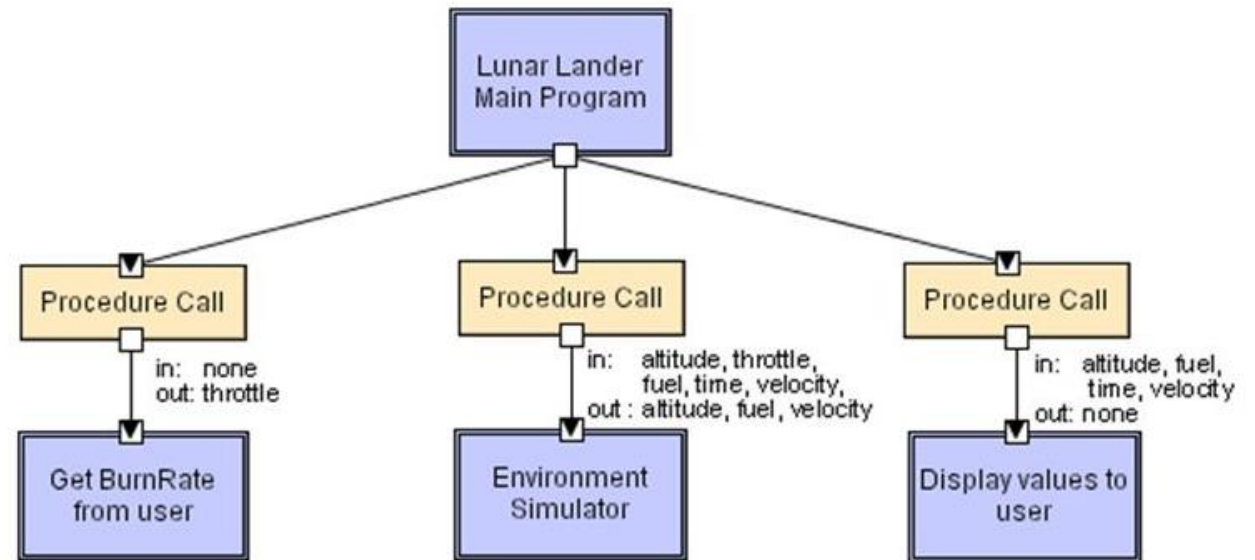
8. P2P (Peer to Peer)

9. Derivados

- C2, Corba

Main y subrutinas: Descripción

- Arquitectura históricamente relevante
- Subrutinas individuales, encapsuladas
- Main "cede" el control a estas subrutinas y determina el orden
- Alto acoplamiento y bloqueante



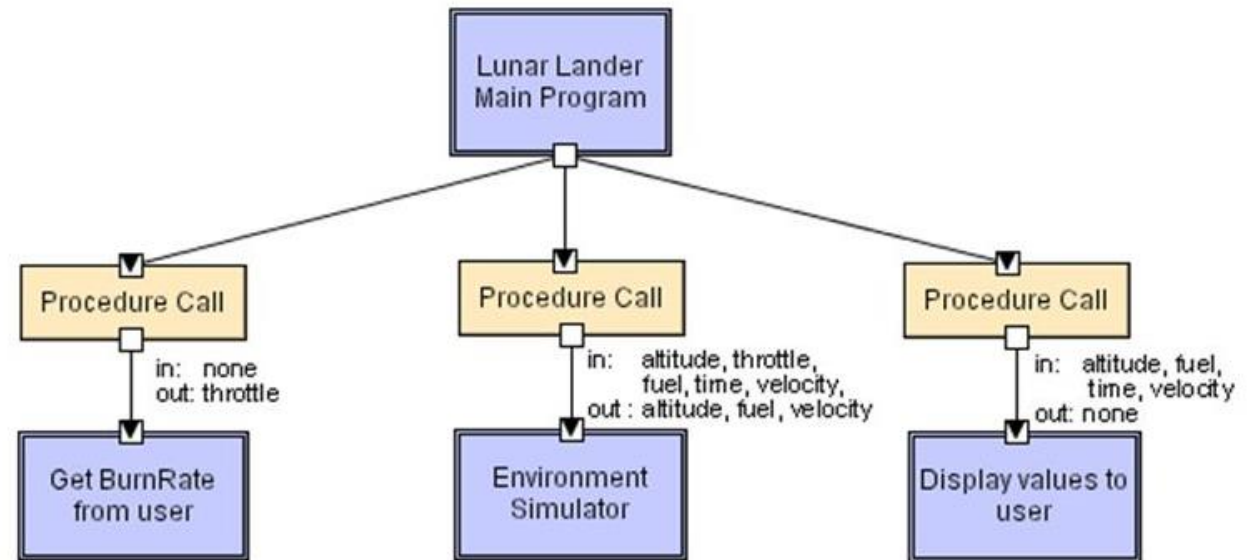
Main y subrutinas: Ejemplo Lunar Lander

- Solo se realiza una acción a la vez
- Otros ejemplos
 - Tetris
 - Sistemas embebidos
 - MS-DOS
 - Pac-Man



Main y subrutinas: Datos y topología

- Datos:
 - Parámetros de llamada de función
- Topología:
 - Centralizado en rutina main: ida y vuelta



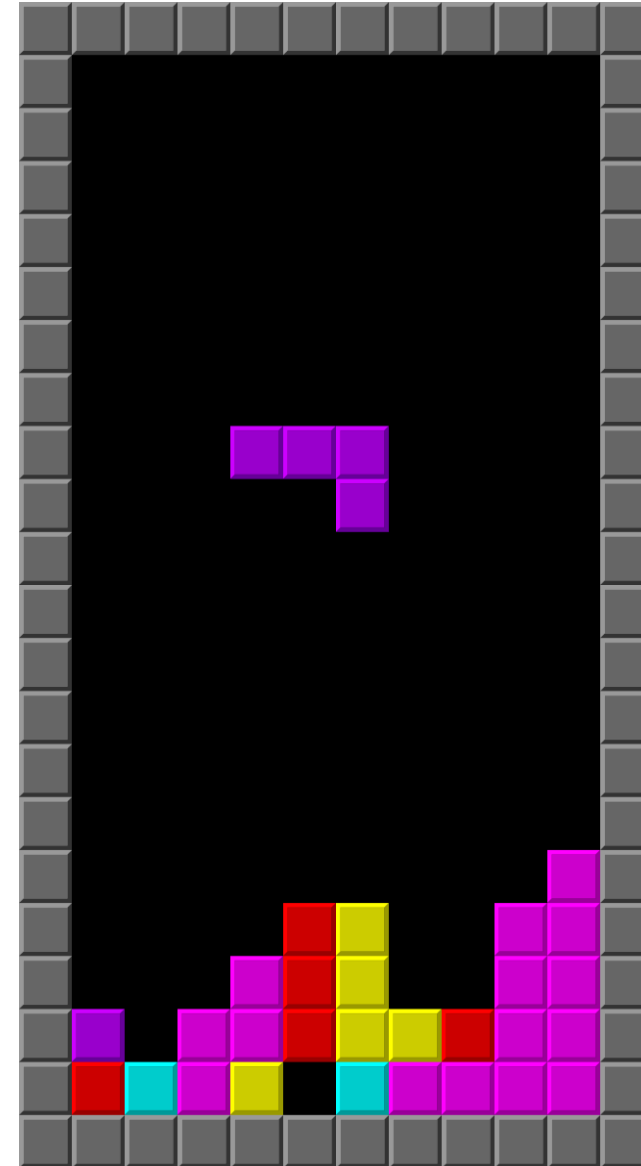
Main y subrutinas: Cualidades

- Modularidad: Si se respeta la interfaz, puedes modificar la subrutina
- Nulo costo de comunicación: *calls* casi instantáneas.
- Varios desarrolladores pueden trabajar en diversas partes de la aplicación (pero no muchos)



Main y subrutinas: Advertencias

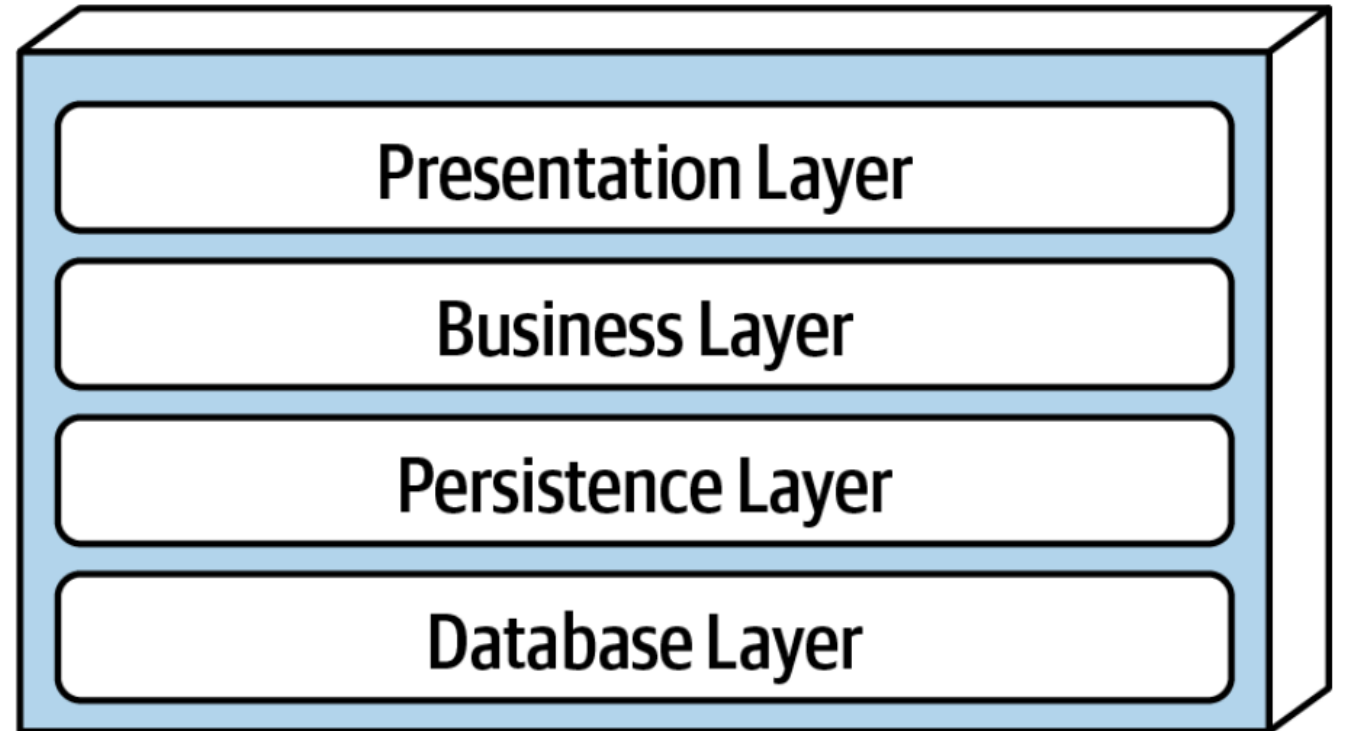
- Pobre extensibilidad y escalabilidad
- Incompatible con estructuras de datos más complejas
- Limitaciones de performance
- Posibilidad de bloqueo



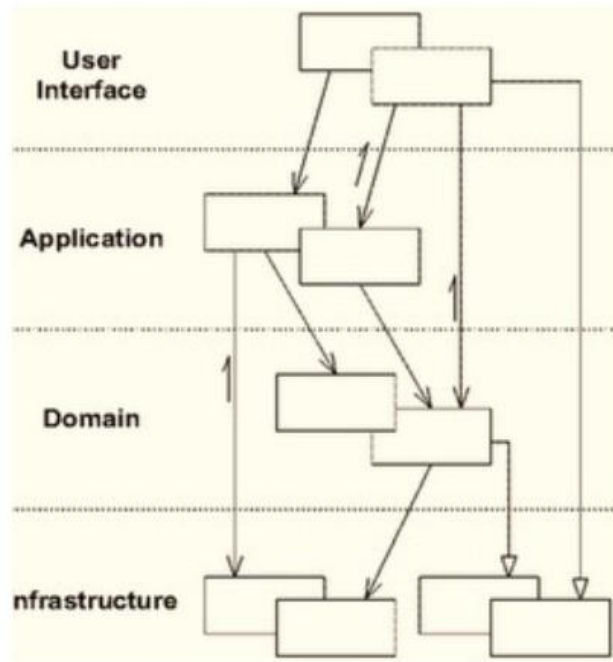
Por capas: Descripción

Organización en **capas lógicas horizontales con roles específicos** en la aplicación

- Estilo muy común
 - Suele salir de una división técnica
- Capas fuertemente ordenadas y cerradas
 - Cerrado: Solo puedes interactuar con capas adyacentes
 - Abierto: Puedes interactuar entre capas
- Cada capa encapsula funcionalidad



Por capas: Ejemplos



Modelo abierto

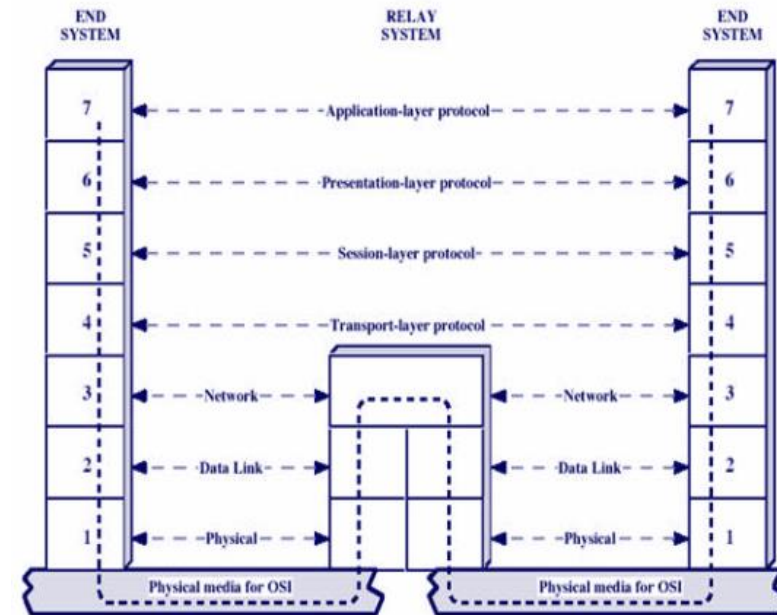


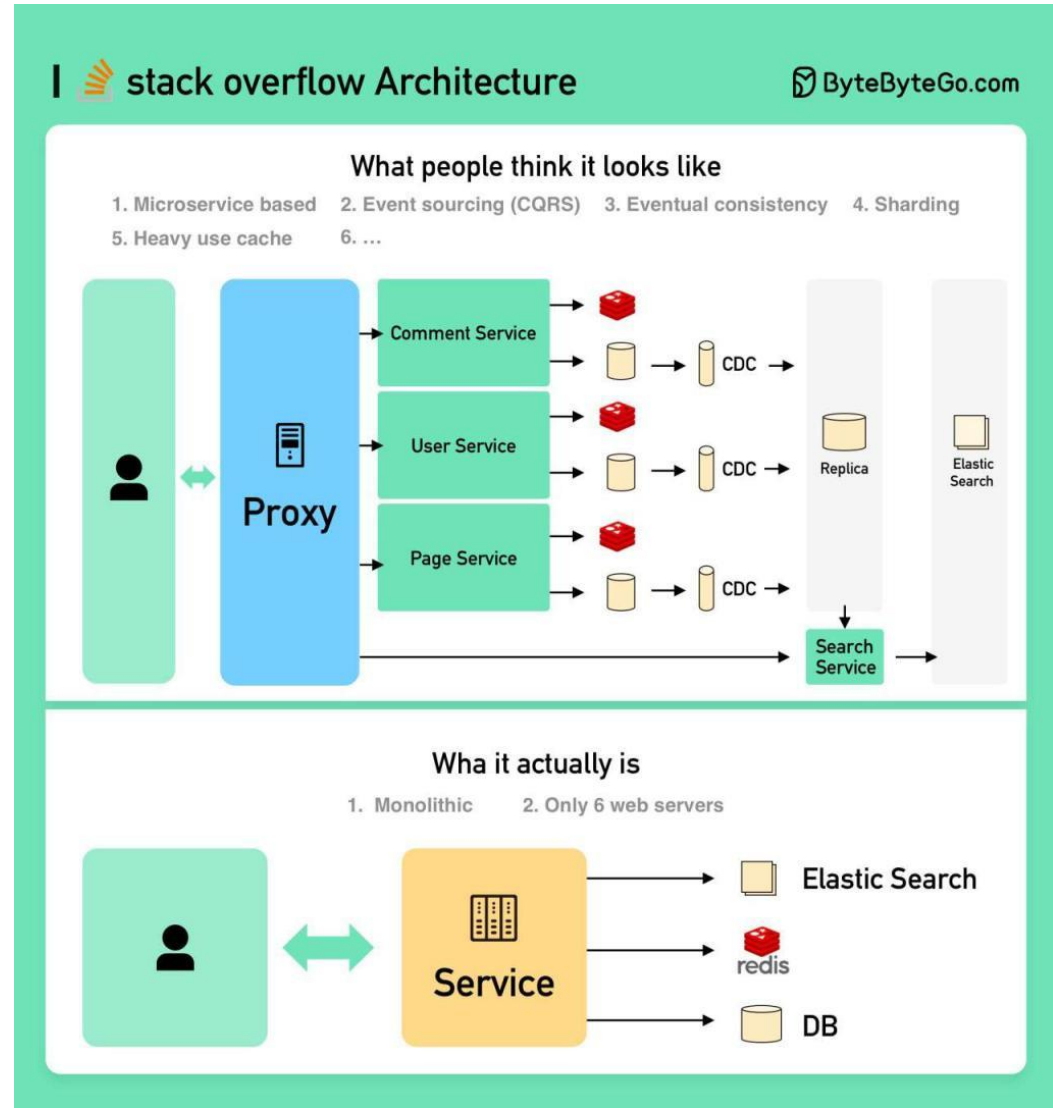
Figure 2.11 The Use of a Relay

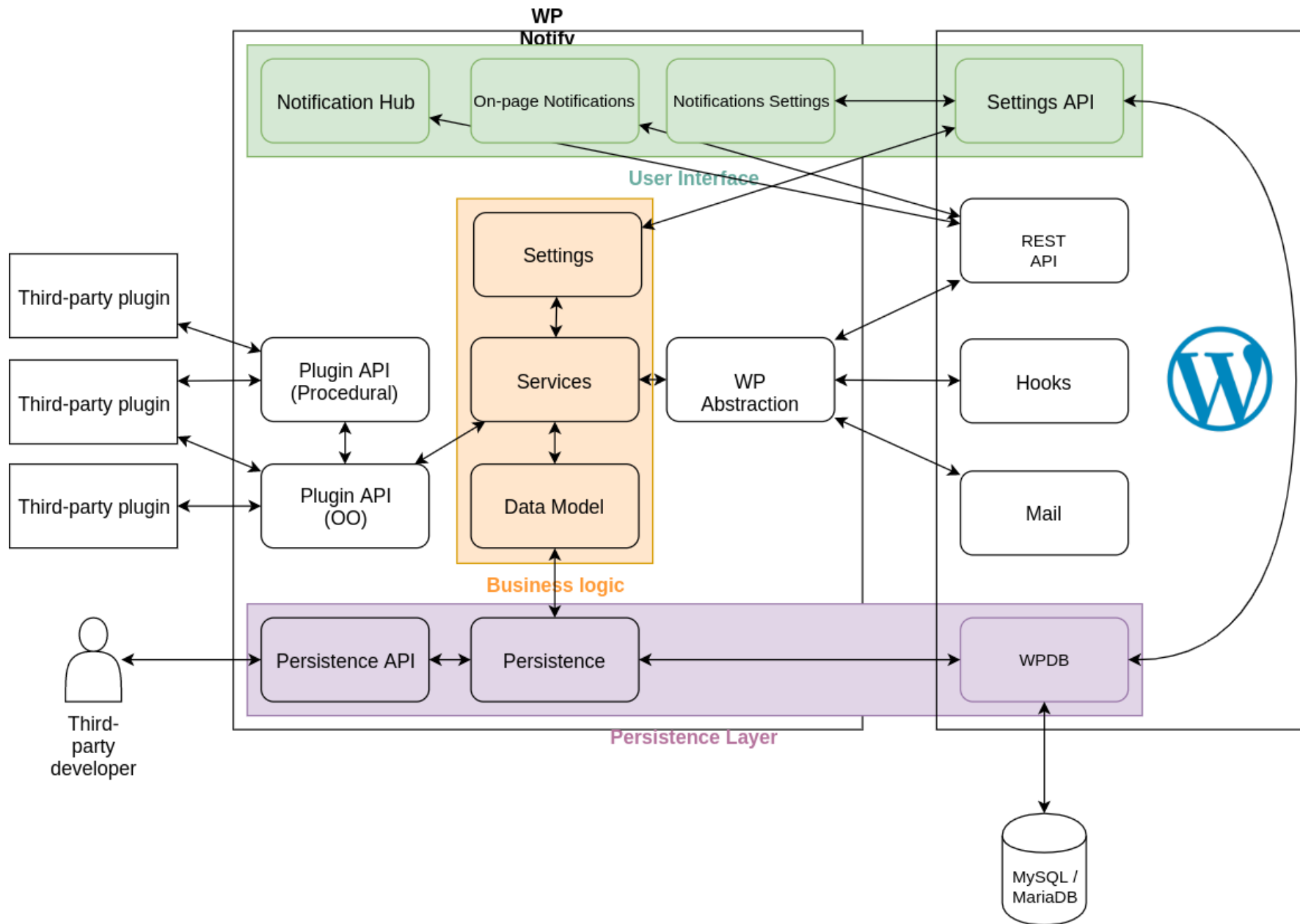
**Modelo cerrado
OSI**

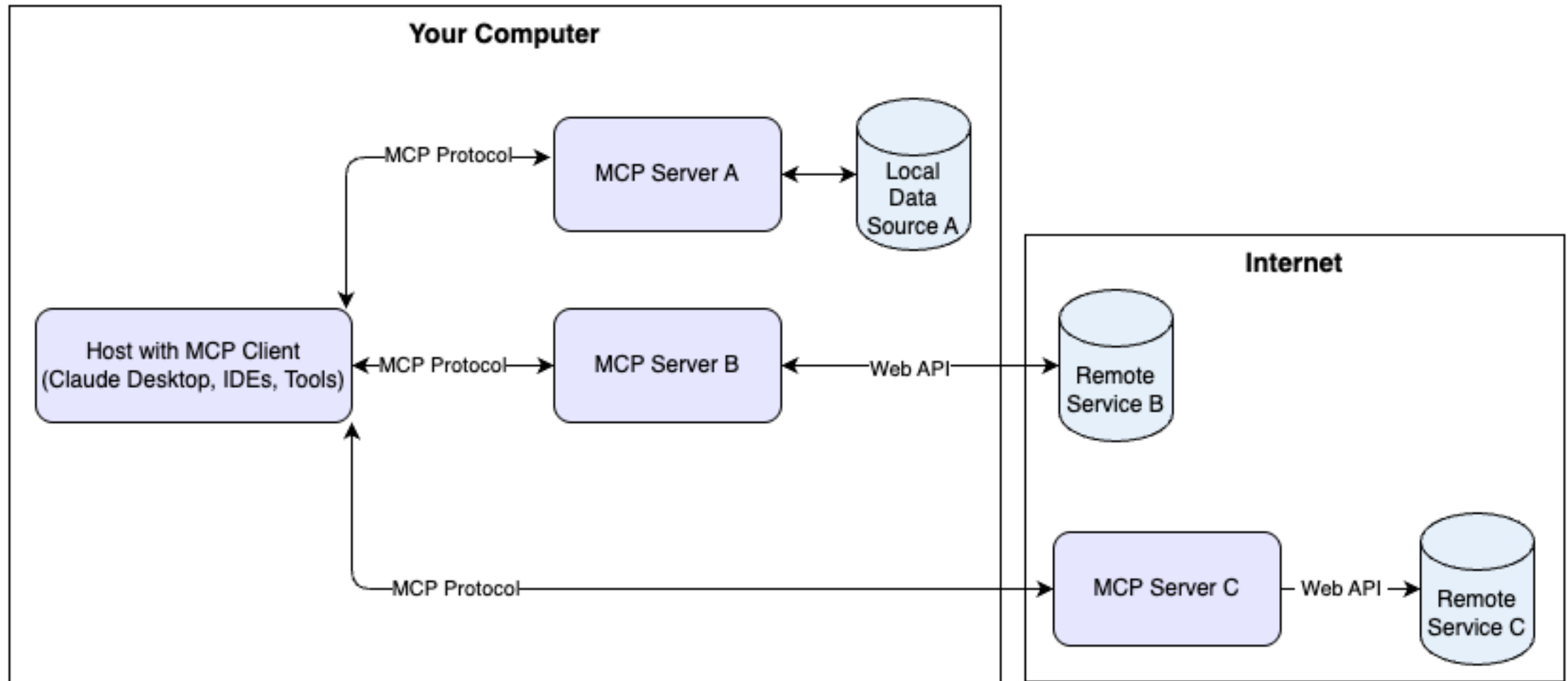
Por capas: Ejemplos

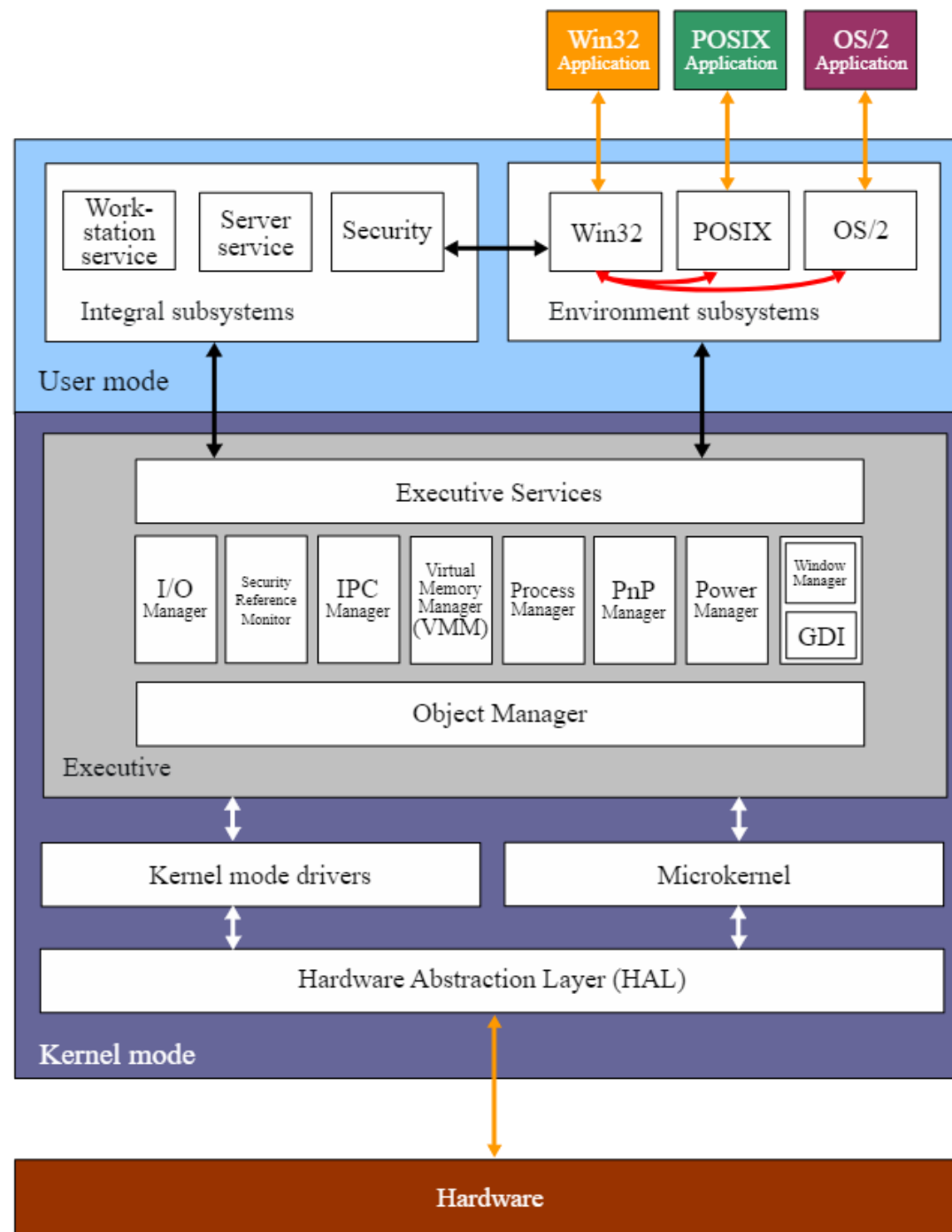
<https://stackoverflow.com/performance>

- Rails
- Máquinas virtuales
- Servicios web
- E-commerce
- Ciertos Kernels de sistema
 - Linux, Windows









Windows - NT Kernel

Linux kernel SCI (System Call Interface)

I/O subsystem

Linux kernel

Virtual File System

Terminals

Sockets

File systems

Netfilter / Nftables

Network
protocols

Linux kernel
Packet Scheduler

Network
device
drivers

Generic
block layer

Linux kernel
I/O Scheduler

Block
device
drivers

Line
discipline

Character
device
drivers

Memory management subsystem

Virtual
memory

Paging
page
replacement

Page
cache

Process management subsystem

Signal
handling

process/thread
creation &
termination

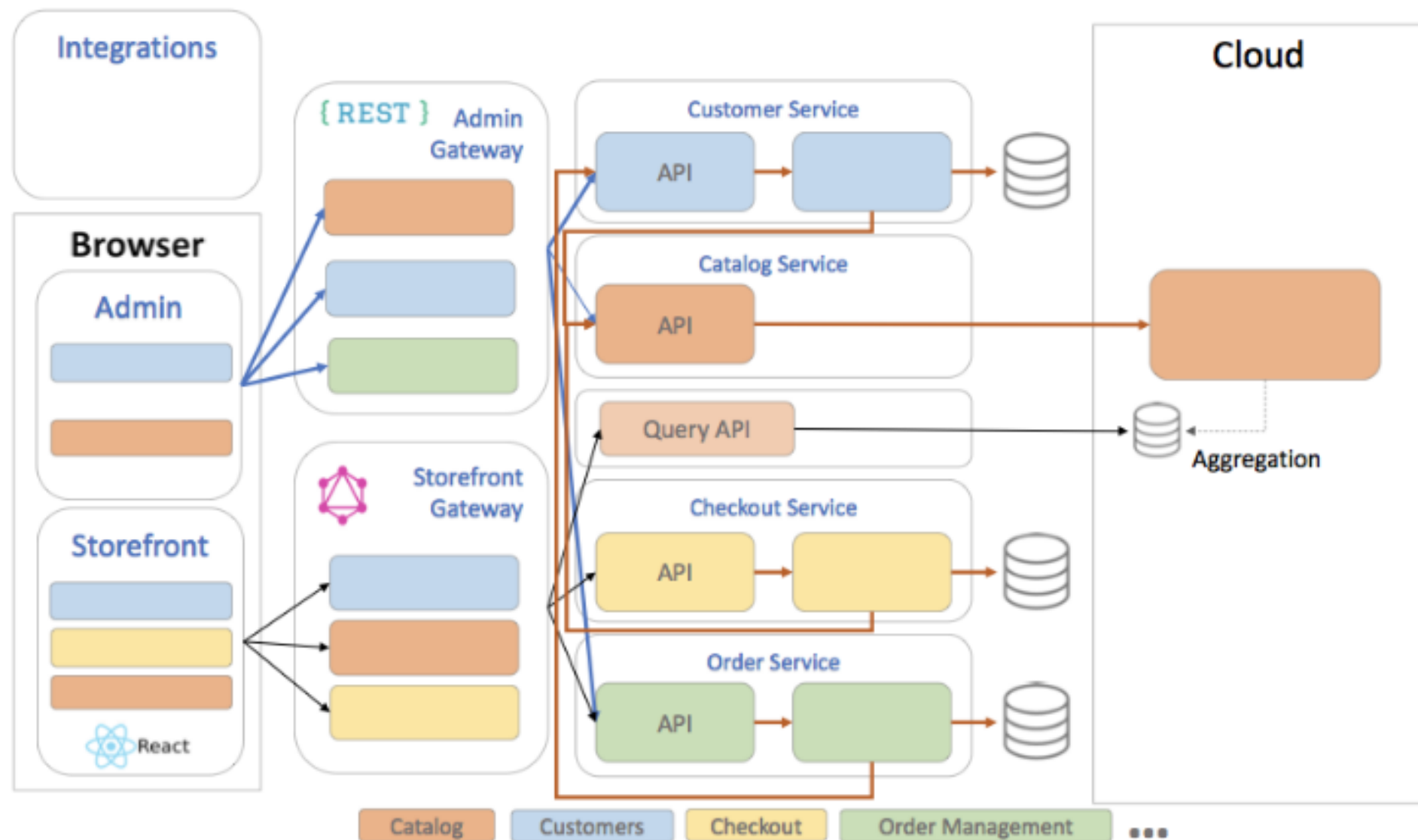
Linux kernel
Process
Scheduler

IRQs

Dispatcher

Desired state

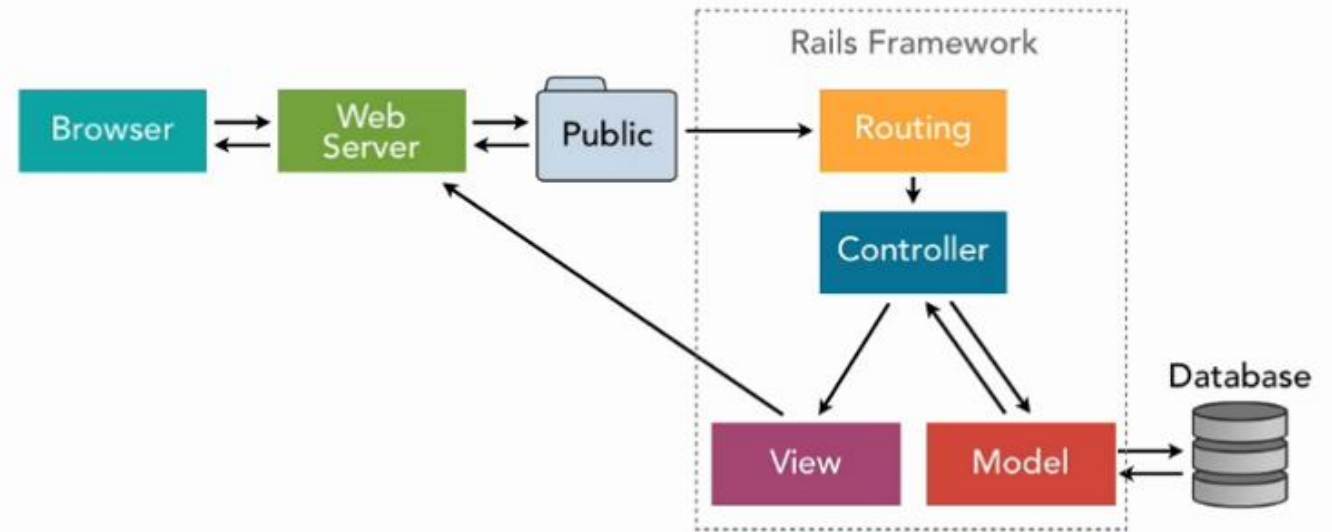
The application consists of technically independent, separately and/or jointly deployable\scalable\replaceable\maintainable parts - services.



Por capas: Datos y topología

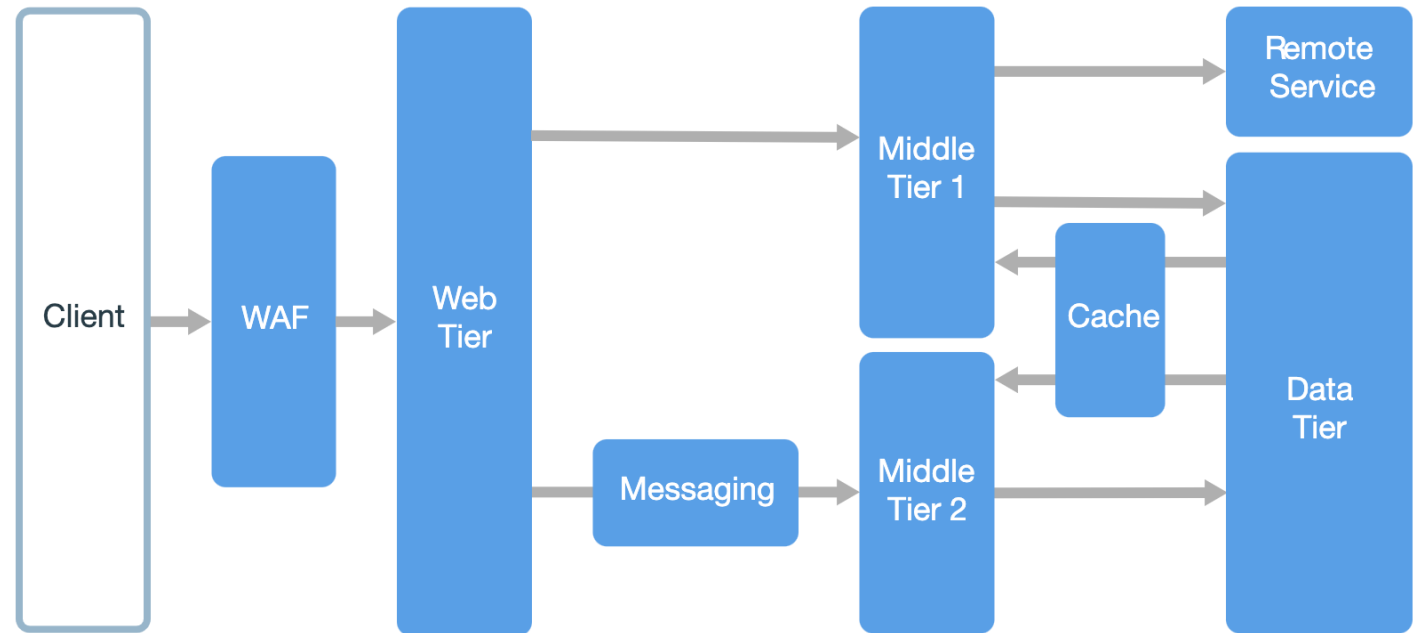
- Datos: Parámetros y paquetes
 - Bien específicos, concisos
 - Interfaz demanda estructura específica
- Topología
 - Lineal: datos se mueven en direcciones predecibles

Rails architecture



Por capas: Pros

- Estilo muy conocido
 - Fácil de diseñar y entender
 - Suele reflejar estructuras de negocio
- Aumenta niveles de abstracción
 - Dependencias muy claras
 - Permite cambiar cosas en una capa solo afectando capas adyacentes
- Interfaces precisas
- Económico de crear



Por capas: Cons

- Pobre desempeño a gran escala
 - Bajo paralelismo comparativamente
- Un exceso de capas reduce performance
- Algunos diseños pueden bloquearse con respuestas
- No es universalmente aplicable
- Difícil evolucionar la arquitectura debido a la rigidez
- Poco tolerante a fallos
- Pueden originarse tiers anémicos





Introducción a Estilos y Patrones

IIC2173 - Arquitectura de sistemas de software

Departamento de Ciencia de la Computación

Escuela de Ingeniería – PUC

Nicolás Acosta – Gerardo Crot